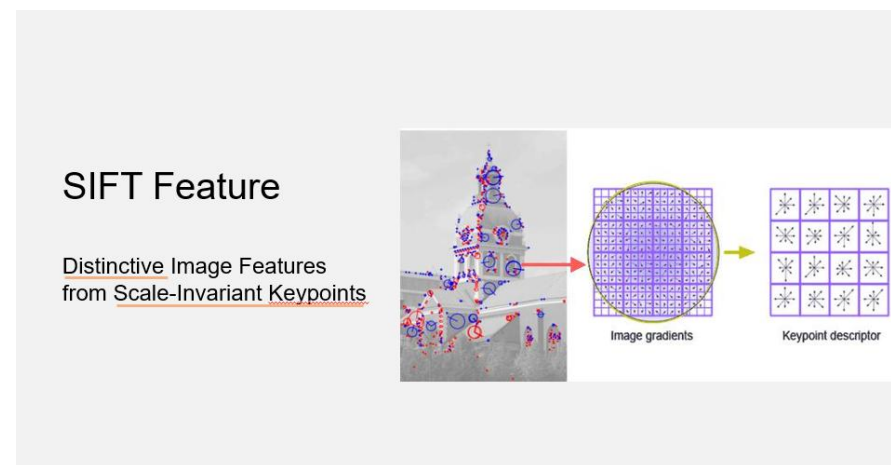
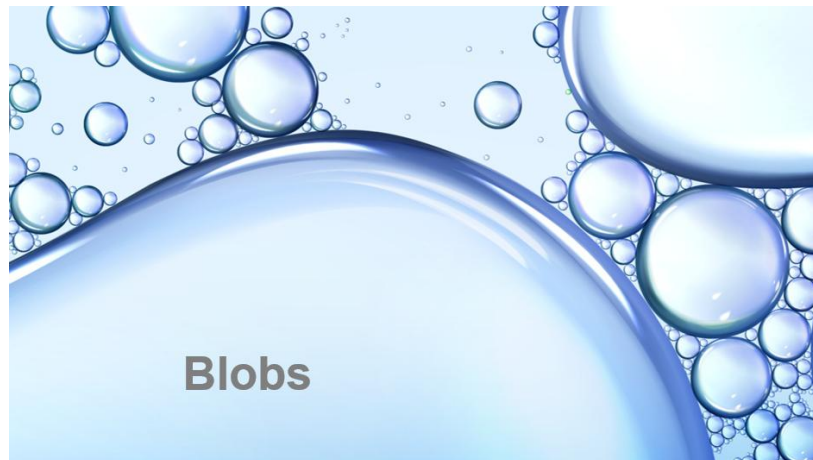


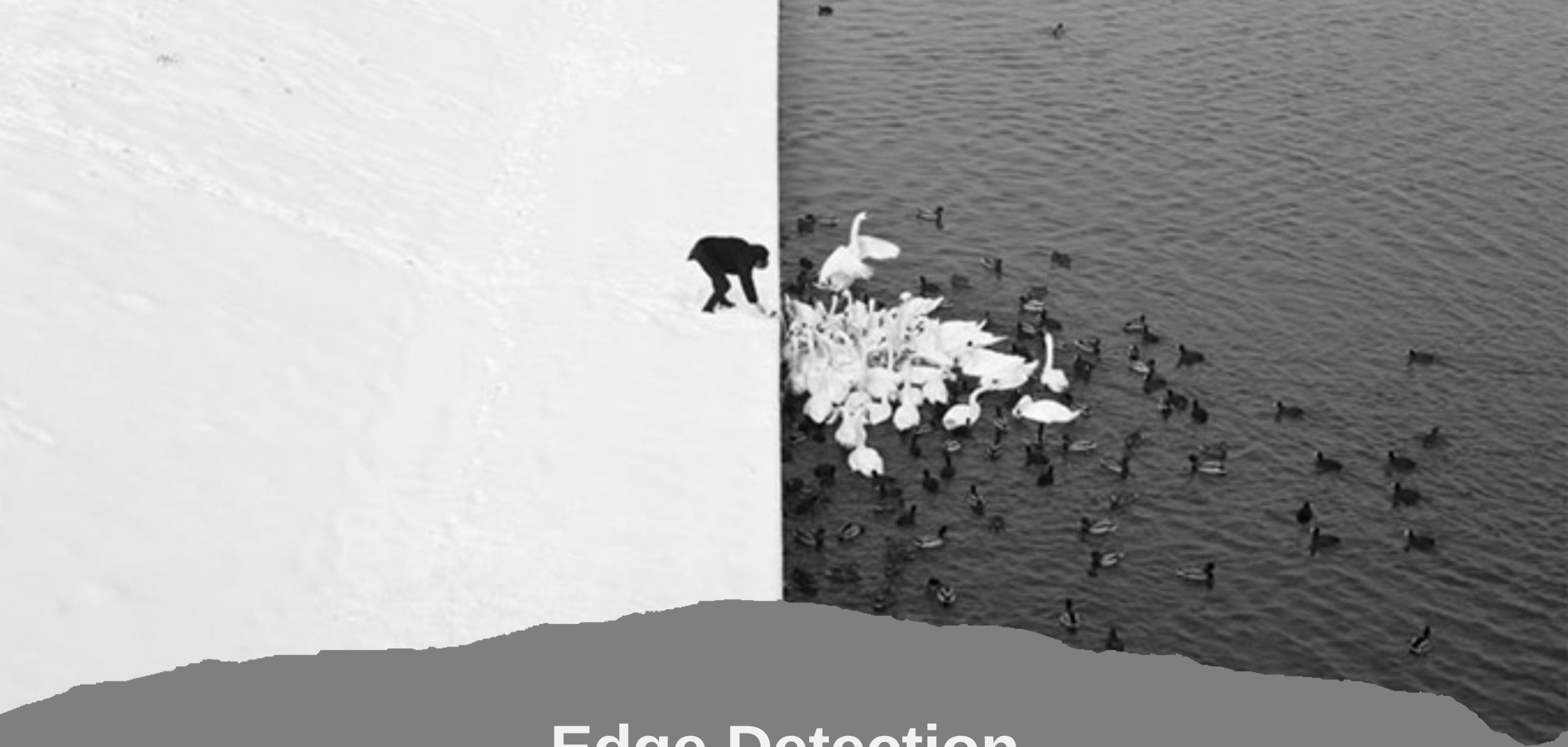
Computer Vision

04 Feature Detection

Peng-Shuai Wang Assistant Professor
Peking University

Contents

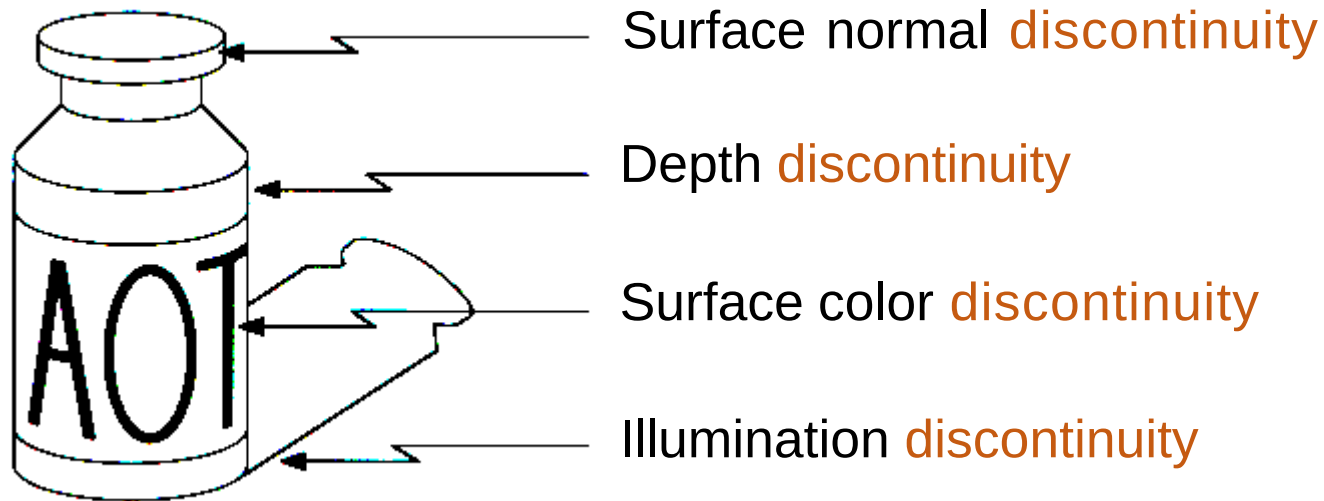




Edge Detection

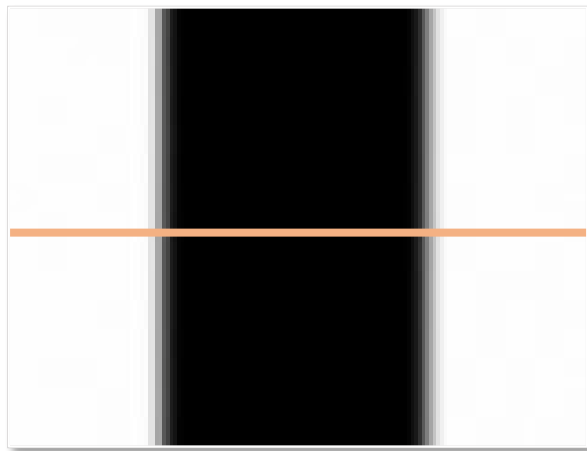
Edge and Edge Detection

- Intuitively, edges carry most of the semantic and shape information from the image
- Edge Detection: Identify sudden changes (discontinuities) in an image
- How to measure the “sudden changes” in an image?

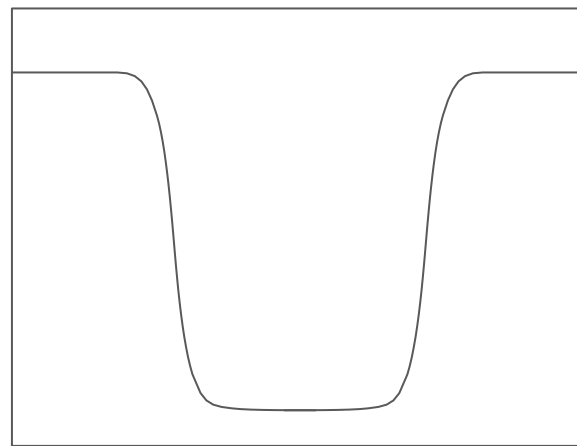


Edge Detection

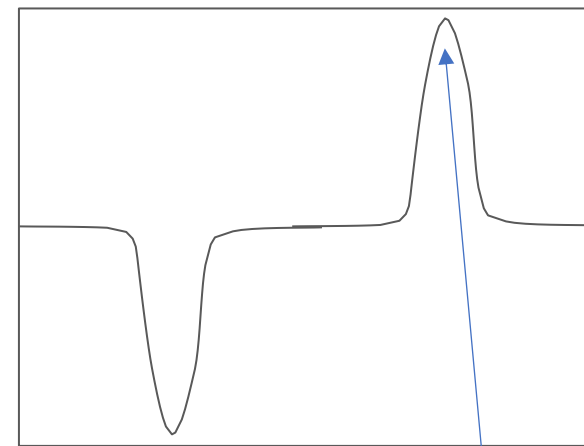
- Derivative: rate at which a function $f(x)$ changes at a point along a direction
- Gradient: all of the partial derivatives stacked together



Image



Intensity function
along horizontal scanline



Derivative

Edges correspond to
extrema of derivative

Partial Derivatives of an Image

- An Image is a 2D function $f(x,y)$

$$f(x + h) = f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \dots$$

X partial derivative: $\frac{\partial f(x, y)}{\partial x} = \lim_{\epsilon \rightarrow 0} \frac{f(x + \epsilon, y) - f(x, y)}{\epsilon}$

Forward differencing: $\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$

-1	1
----	---

Central differencing: $\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x - 1, y)}{2}$

-1	0	1
----	---	---

Partial Derivatives of an Image

- Partial Derivatives of an Image can be computed as image filtering

Other Differentiation
Operations

Horizontal

Vertical

Prewitt

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

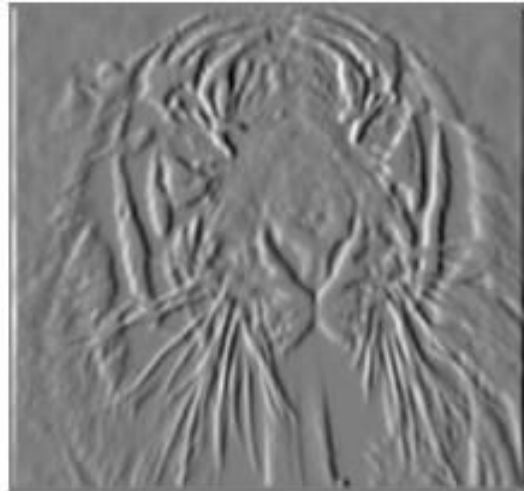
$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Sobel

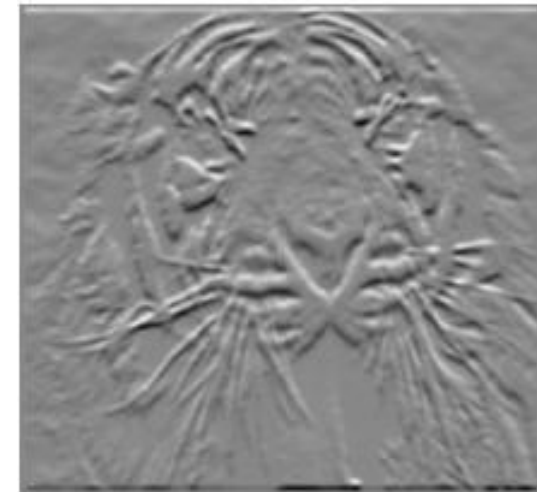
$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Partial derivatives of an image



-1	1
----	---



-1
1

Noise Severely Affects Derivatives of an Image

Smooth image + filter with

-1	0	1
----	---	---

$$I_{i,j} = \text{True image} \quad \epsilon_{i,j} \sim N(0, \sigma^2)$$

$$D_{i,j} = (I_{i,j+1} + \epsilon_{i,j+1}) - (I_{i,j-1} + \epsilon_{i,j-1})$$

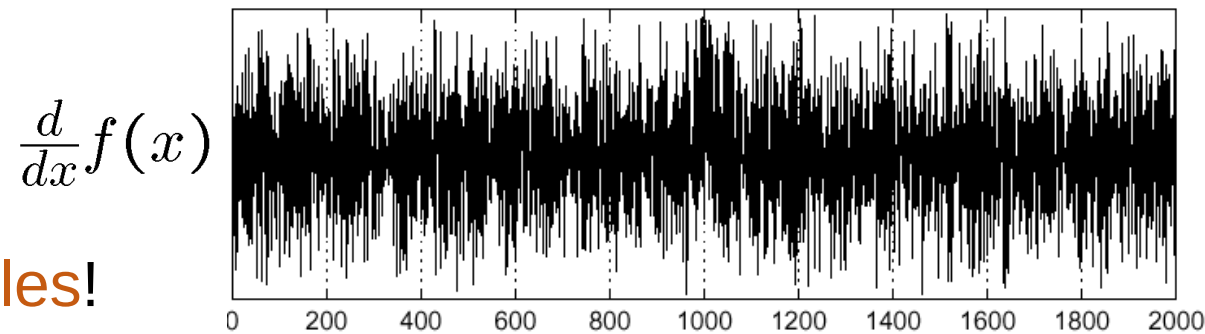
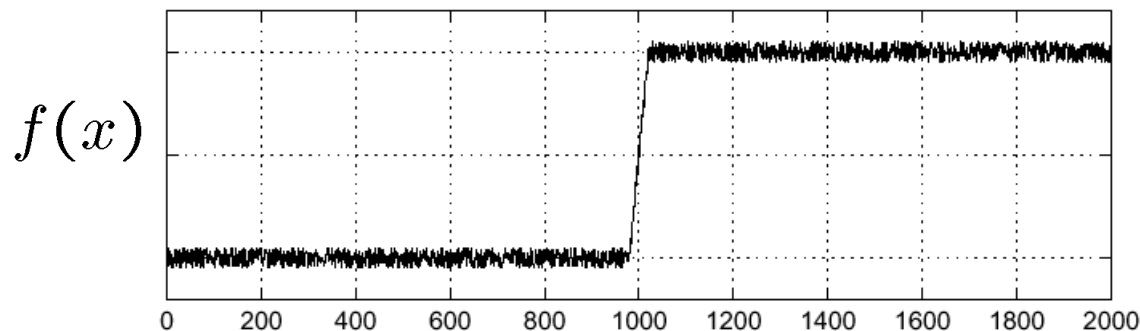
$$D_{i,j} = \underbrace{(I_{i,j+1} - I_{i,j-1})}_{\text{True difference}} + \underbrace{\epsilon_{i,j+1} - \epsilon_{i,j-1}}_{\text{Sum of 2 Gaussians}}$$

True
difference

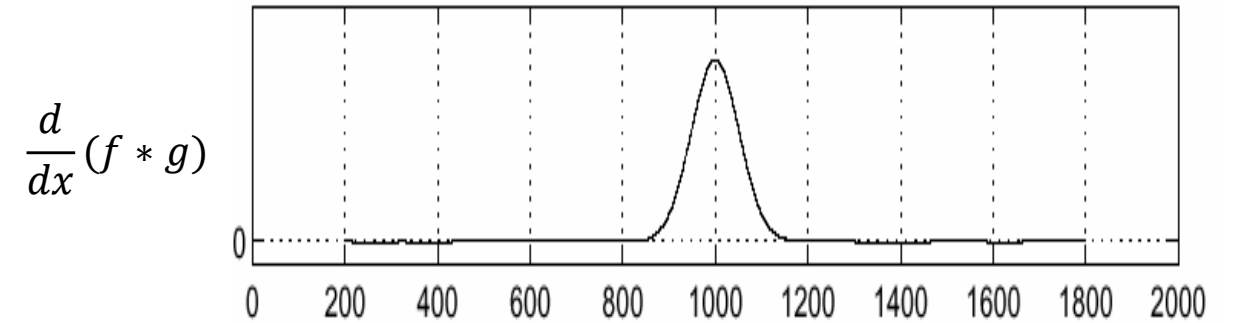
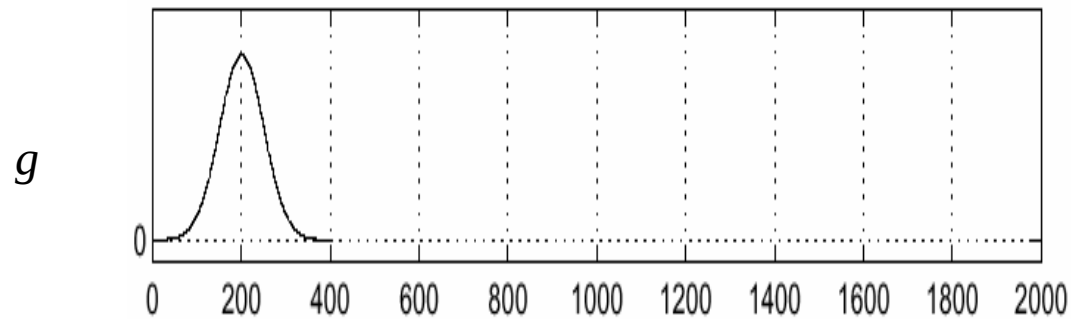
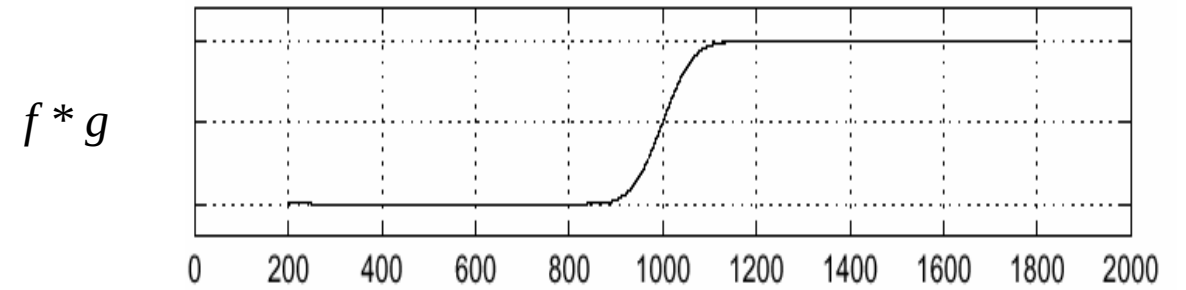
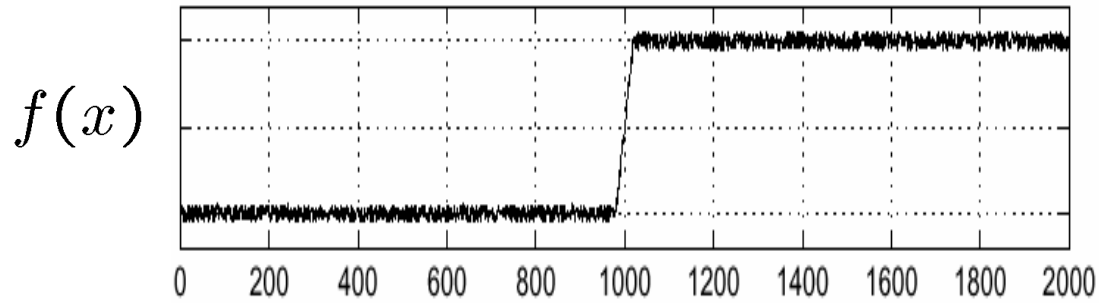
Sum of 2
Gaussians

$$\epsilon_{i,j} - \epsilon_{k,l} \sim N(0, 2\sigma^2) \rightarrow \text{Variance doubles!}$$

Consider a row of $f(x,y)$ (i.e., fix y)



Handle Noise by Prefiltering the Image

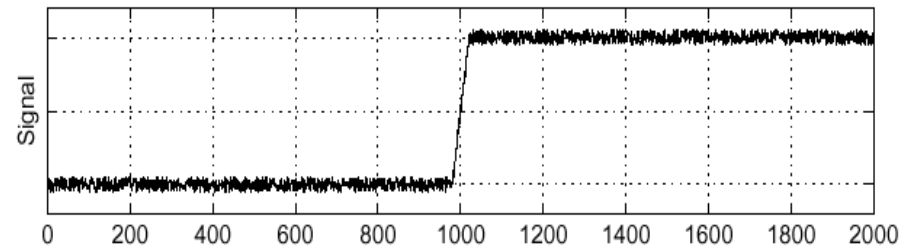


Let's Make It One Pass (1D)

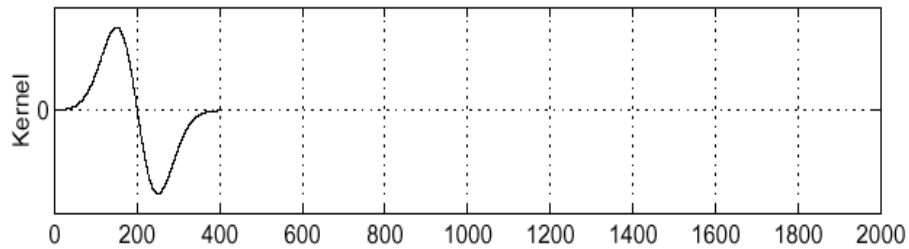
$$\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$$

Sigma = 50

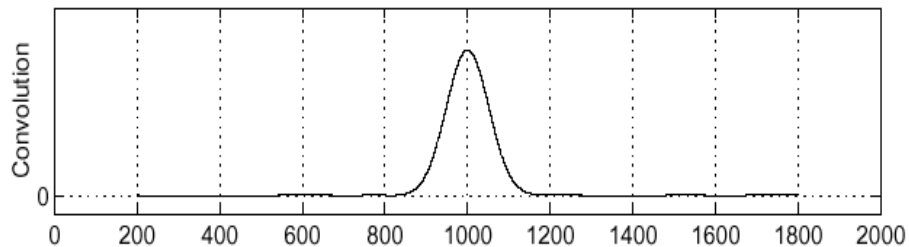
f



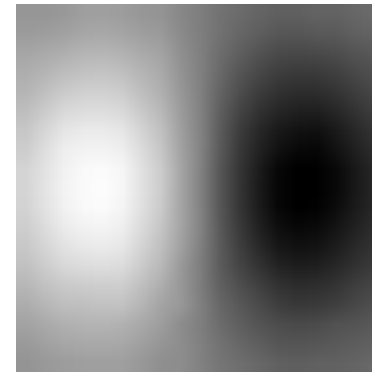
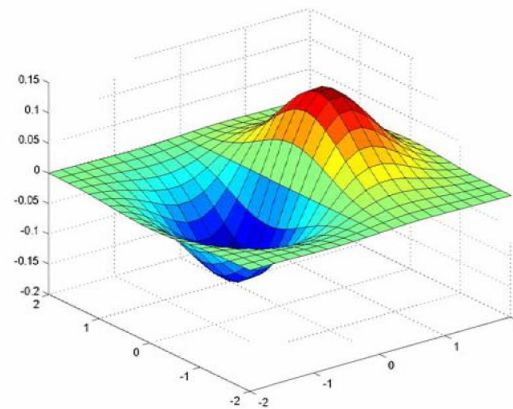
$\frac{d}{dx}g$



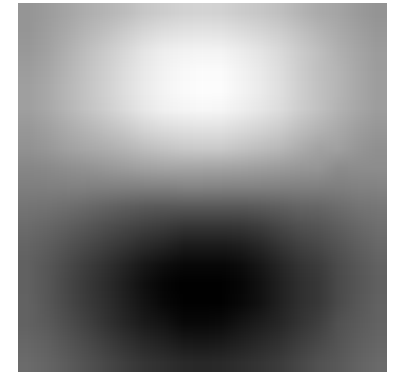
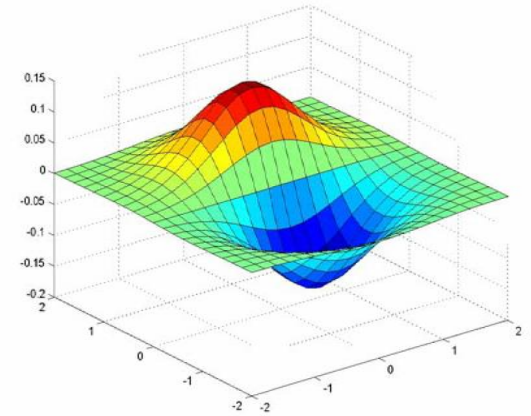
$f * \frac{d}{dx}g$



$\frac{\partial}{\partial x}G_\sigma$

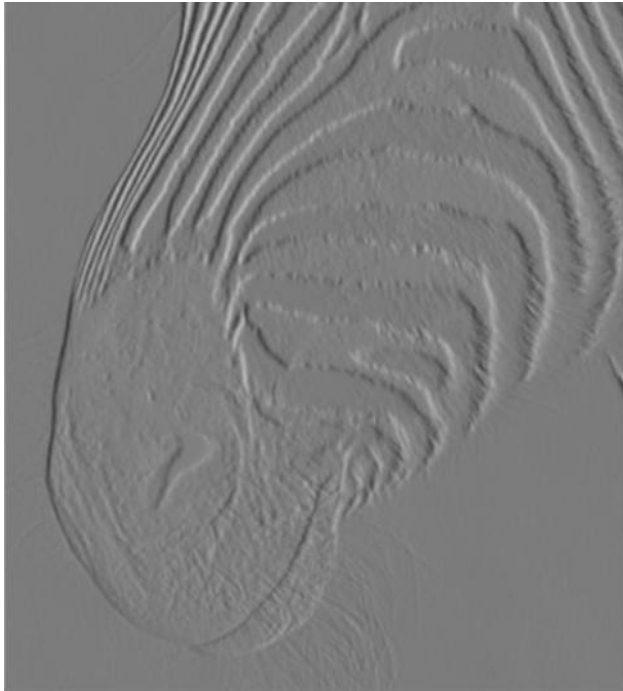


$\frac{\partial}{\partial y}G_\sigma$

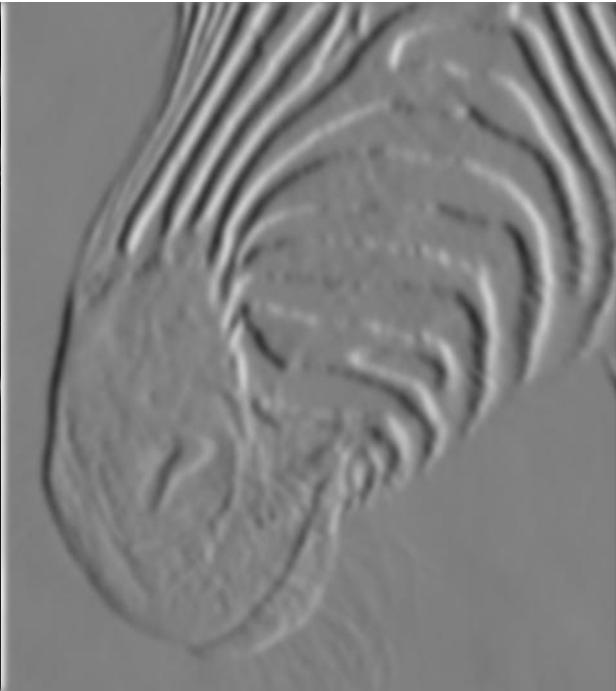


Applying the Gaussian Derivative

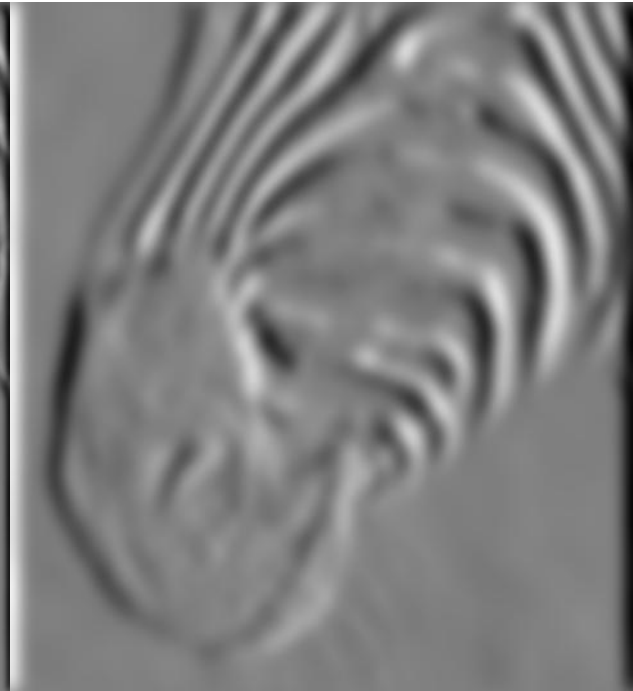
1 pixel



3 pixels



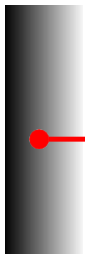
7 pixels



Remove noise, but blur edges

Image Gradient

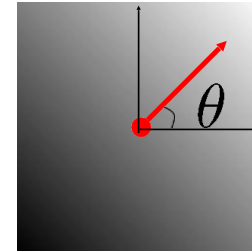
The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$



$$\nabla f = \left[\frac{\partial f}{\partial x}, 0 \right]$$



$$\nabla f = \left[0, \frac{\partial f}{\partial y} \right]$$



$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

The gradient points in the direction of most rapid increase of intensity:

The gradient direction is given by: $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

The edge strength is given by the **gradient magnitude**: $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$

Canny Edge Detector

1. Preprocessing: Grayscale conversion + Gaussian blur



Input Image



Gray Image



Gaussian Blur

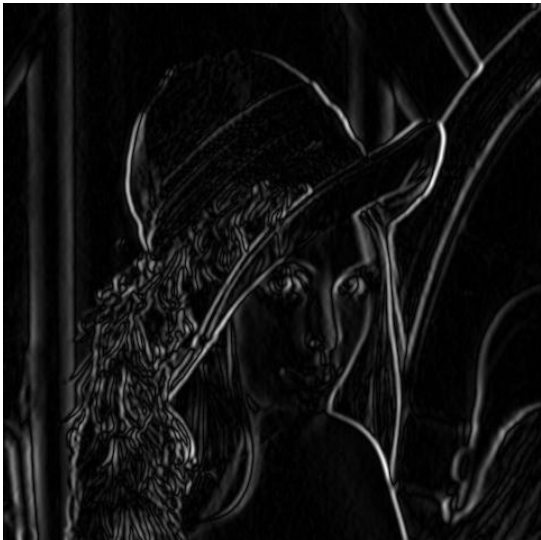
Canny Edge Detector

2. Use **central differencing** to compute gradient image (instead of first forward differencing), which is more accurate.

3. **Calculate gradient magnitude**

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

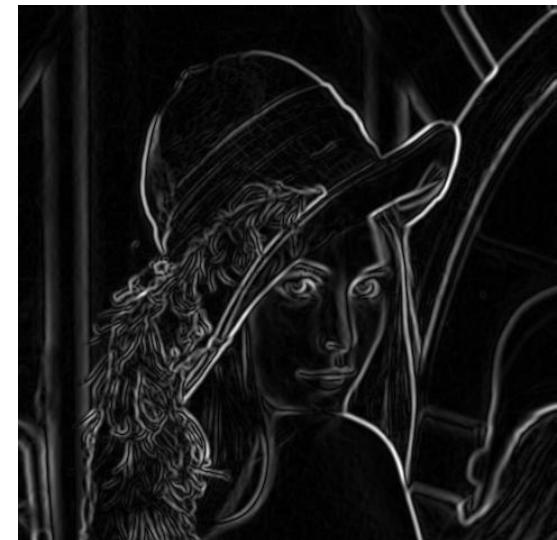
$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$



Gx



Gy



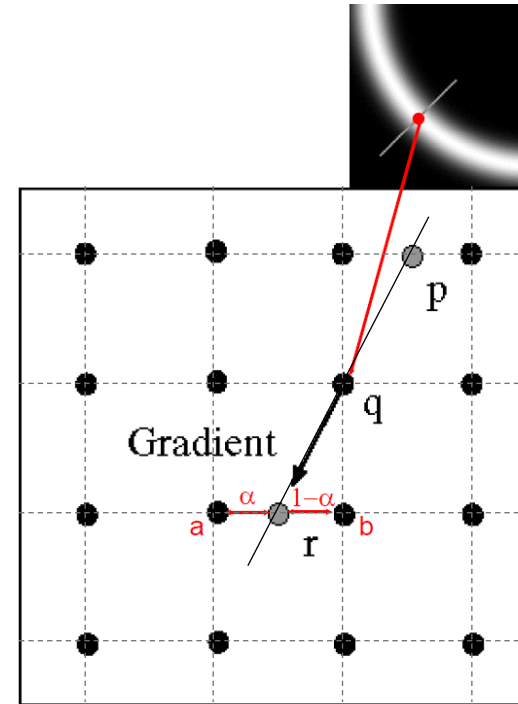
Gradient Magnitude

Sobel Edge Detector thresholds the gradient magnitude to obtain edges. But thresholds are brittle, and the thresholds are hard to choose.

Canny Edge Detector

4. Non-Maximum Suppression

- The gradient magnitude produced results in thick edges. We perform non-maximum suppression to thin out the edges.
- Go through all the points on the gradient intensity matrix, **find and only keep** the pixels with the maximum value in the edge directions.
- Example: keep q if $q > p$ and $q > r$



$$r = \alpha b + (1 - \alpha)a$$

Edges are too thick,
make them thinner.



Non Maximum Suppression

Canny Edge Detector

5. Double threshold

- Pixels with a high value (e.g., 0.7) are most likely to be edges.
- Pixels with a low value (e.g., 0.3) are most likely not to be edges.
- Other pixels are defined as weak edges
- Weak edges are further classified as edge or not in the next step



Non Maximum Suppression



Double Thresholding

Canny Edge Detector

6. Edge Tracking by Hysteresis

Weak edges that are connected to strong edges will be actual/real edges.
Weak edges that are not connected to strong edges will be removed.



Input Image



Final Result

Summary of Canny Edge Detector

- 1. Grayscale + Gaussian Filter
- 2. Central differencing
- 3. Calculate gradient magnitude
- 4. Non-Maximum Suppression
- 5. Double threshold
- 6. Edge Tracking by Hysteresis

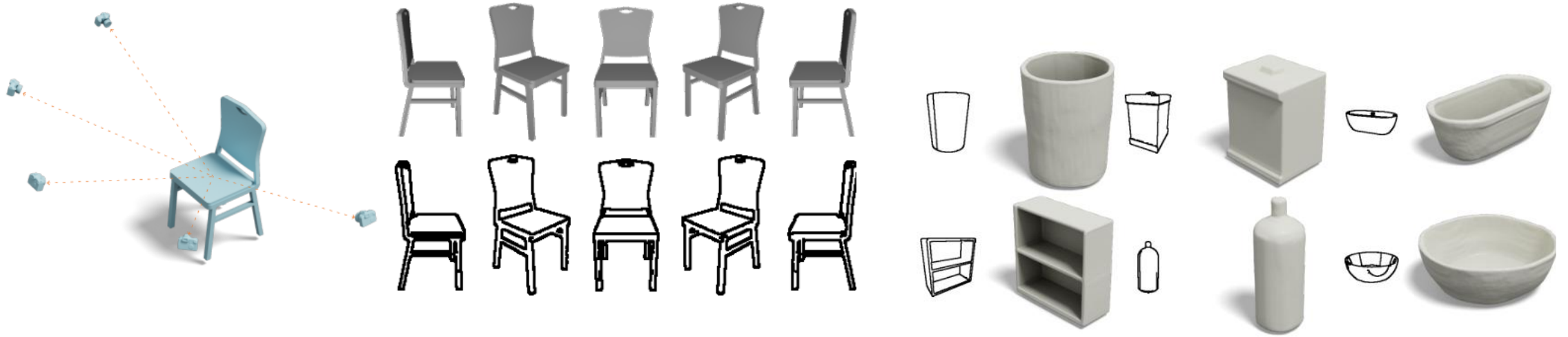


Input Image



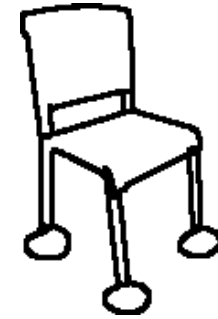
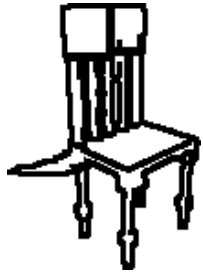
Final Result

Applications of Canny Edge Detector



Use Canny edge detector to generate sketches

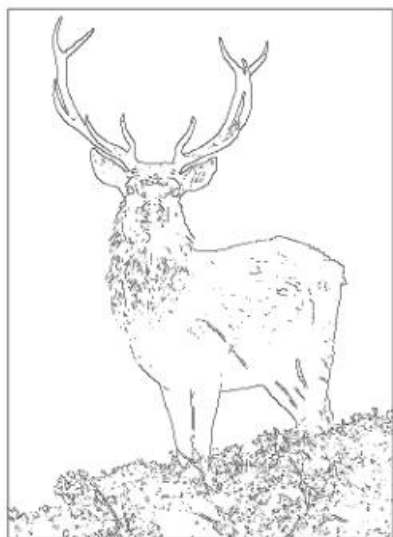
Applications of Canny Edge Detector



Adding Conditional Control to Text-to-Image Diffusion Models

Lvmin Zhang, Anyi Rao, and Maneesh Agrawala
Stanford University

`{lvmin, anyirao, maneesh}@cs.stanford.edu`



Input Canny edge



Default



“masterpiece of fairy tale, giant deer, golden antlers”



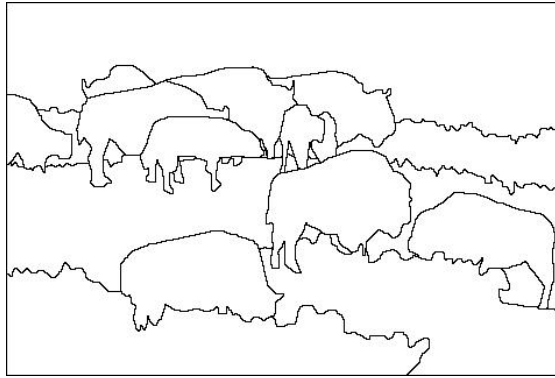
“..., quaint city Galic”

Edge Detection is Still an Unsolved Problem

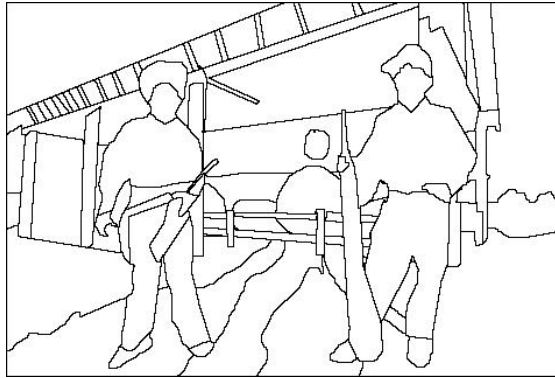
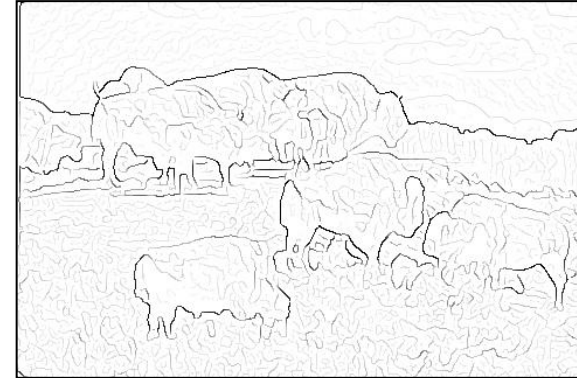
Image



human segmentation



gradient magnitude



Corners



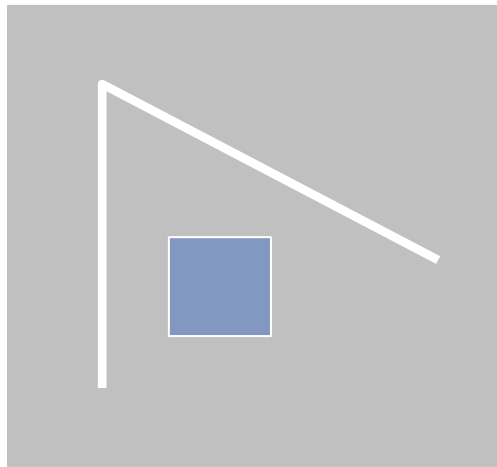
Corners for Localizing Reliably

- To localize a position, edges are not accurate enough.
- We can resort to the intersection of two (or more) edges, i.e., corners.

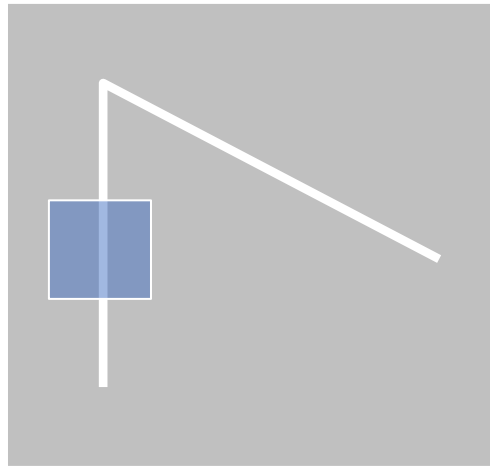


Harris Corner Detector

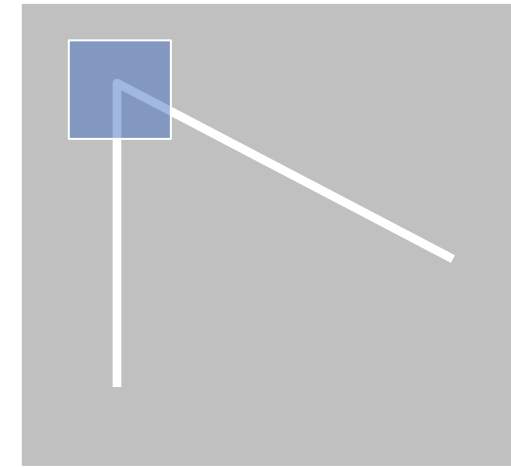
- A corner can be interpreted as the intersection of two (or more) edges
- **Key idea:** characterize corners by measuring the intensity change when shifting a small local window



Flat region: no change
in all directions



Edge: no change along
the edge direction



Corner: significant change
in all directions

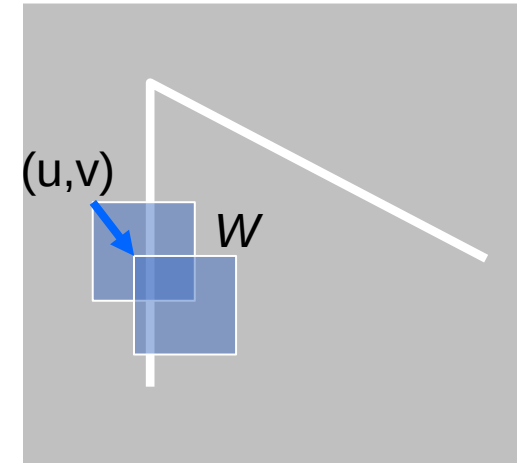
Harris Corner Detector

Measure intensity change when shifting a window

- Consider shifting the window W by (u,v) ,
- The intensity change of each pixel before and after shifting can be defined by summing up the squared differences (SSD)

$$E(u, v) = \sum_{(x,y) \in W} w(x, y) (I(x + u, y + v) - I(x, y))^2$$

- $I(x, y)$ is the pixel value and $w(x, y)$ is the pixel weight
- However, it is slow to compute SSD exactly for each pixel and each offset (u, v)



Taylor Series for Images

Recall Taylor Series – a way of *linearizing* a function:

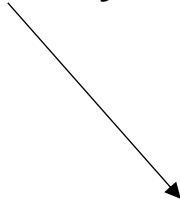
$$f(x + d) \approx f(x) + \frac{\partial f}{\partial x} d$$

Do the same with images, treating them as functions of x, y

$$I(x + u, y + v) \approx I(x, y) + I_x u + I_y v$$

For brevity: $I_x = \frac{\partial I}{\partial x}, I_y = \frac{\partial I}{\partial y}$

(Small motion assumption)

$$E(u, v) = \sum_{(x,y) \in W} w(x, y) (I(x + u, y + v) - I(x, y))^2$$


The Sum of Squared Differences with Taylor Series

$$\begin{aligned} E(u, v) &= \sum_{(x,y) \in W} \left(I(x+u, y+v) - I(x, y) \right)^2 \rightarrow w(x, y) \text{ is set to 1 for simplicity} \\ &\approx \sum_{(x,y) \in W} \left(\underline{I(x, y)} + I_x u + I_y v - \underline{I(x, y)} \right)^2 \\ &= \sum_{(x,y) \in W} \left(I_x u + I_y v \right)^2 \\ &= \sum_{(x,y) \in W} I_x^2 u^2 + 2I_x I_y uv + I_y^2 v^2 \\ &= [u, v] \mathbf{M} [u, v]^T \end{aligned}$$
$$\mathbf{M} = \begin{bmatrix} \sum_{x,y \in W} I_x^2 & \sum_{x,y \in W} I_x I_y \\ \sum_{x,y \in W} I_x I_y & \sum_{x,y \in W} I_y^2 \end{bmatrix}$$

$\rightarrow \mathbf{M}$ is called the second moment matrix

Review Quadratic Forms

Suppose have symmetric matrix $\mathbf{M}$ and a vector $[u, v]$:

$$E([u, v]) = [u, v]\mathbf{M}[u, v]^T$$

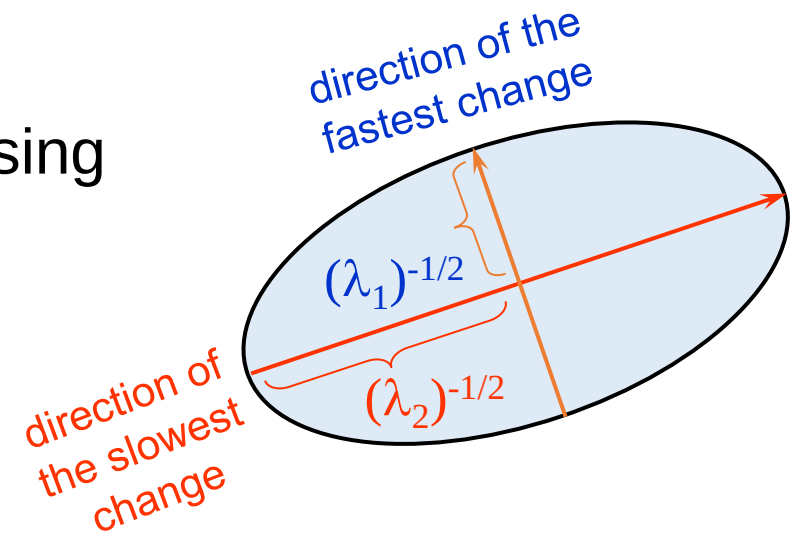
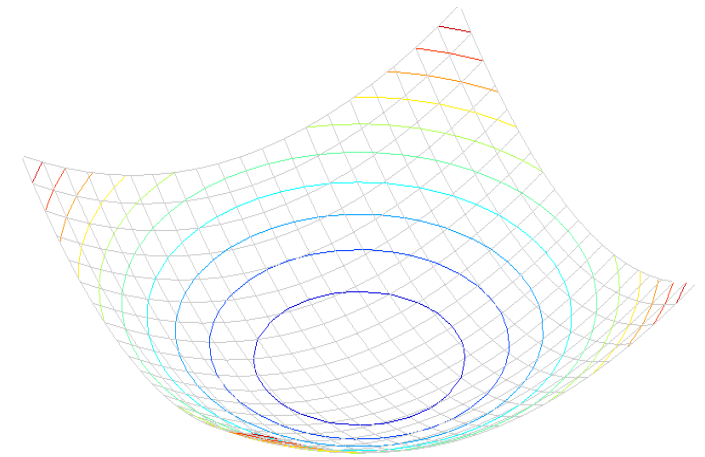
Then the isocontour / slice-through of F , i.e.

$$E([u, v]) = a$$

is an ellipse.

We can look at the shape of this ellipse by decomposing $\mathbf{M}$ into a rotation + scaling

$$\mathbf{M} = \mathbf{R}^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \mathbf{R} \quad \lambda_1 \text{ and } \lambda_2 \text{ are eigenvalues}$$



Interpreting Eigenvalues of M

$$R = \det(\mathbf{M}) - \alpha \operatorname{trace}(\mathbf{M})^2$$

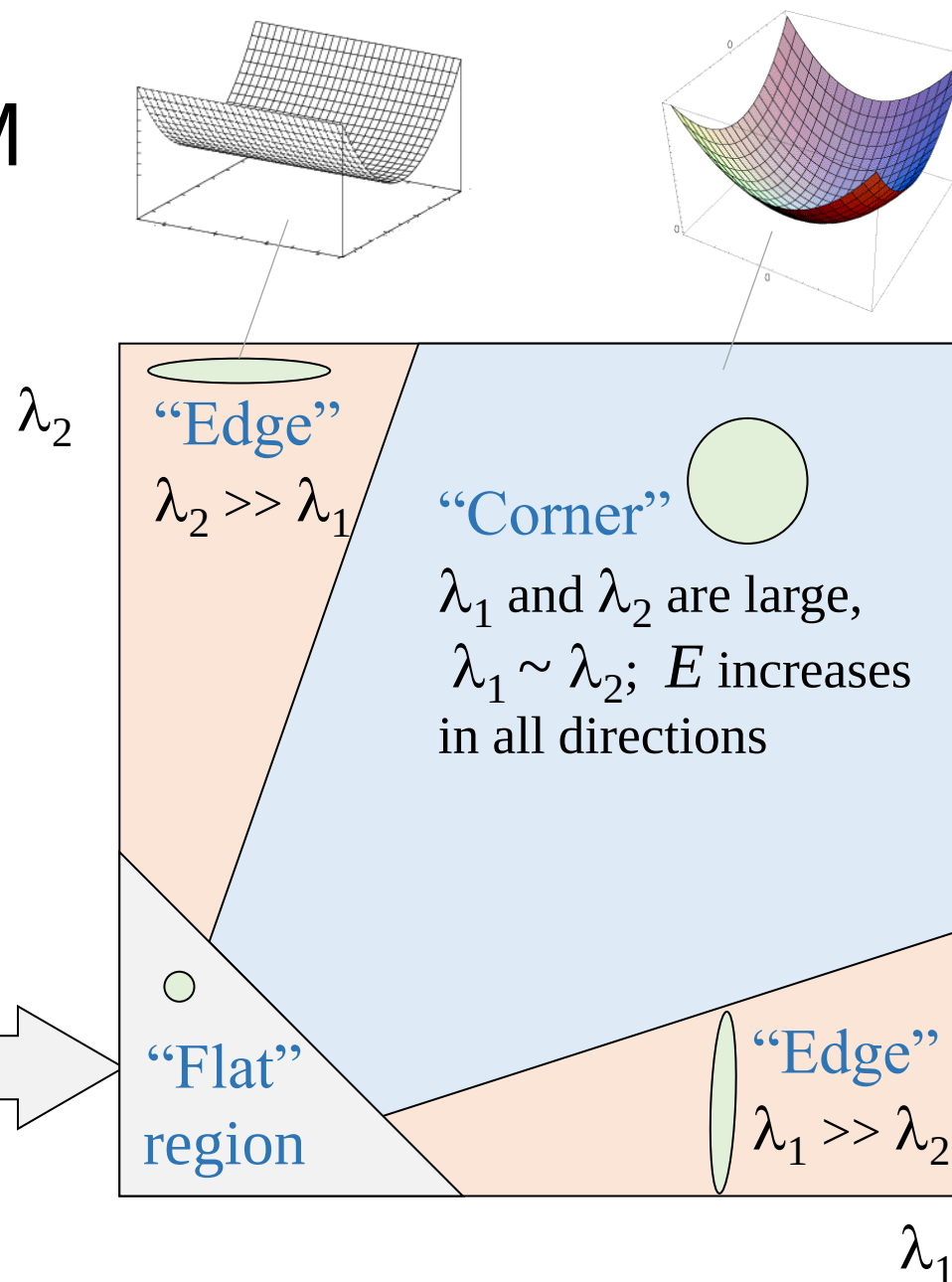
$$= \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

α : constant (0.04 to 0.06)

$$R = \frac{\det(\mathbf{M})}{\operatorname{trace}(\mathbf{M}) + \epsilon} = \frac{\lambda_1 \lambda_2}{\lambda_1 + \lambda_2 + \epsilon}$$

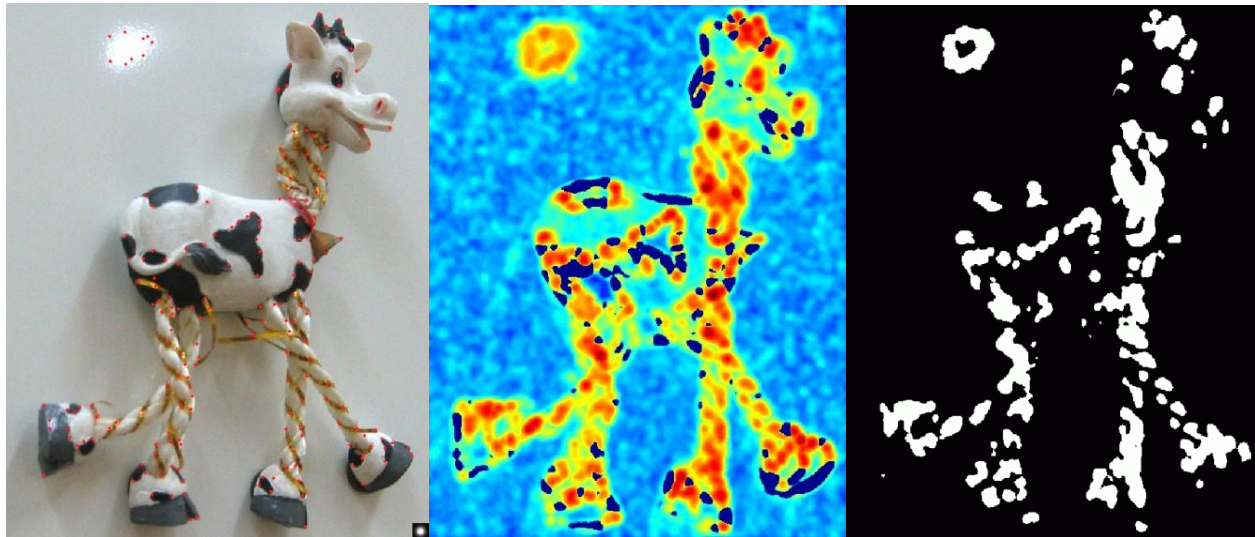
Calculating eigenvalues is usually slow! Determinant and trace have closed-form solutions (?).

λ_1 and λ_2 are small;
 E is almost constant
in all directions



Summary of **Harris** Corner Detector

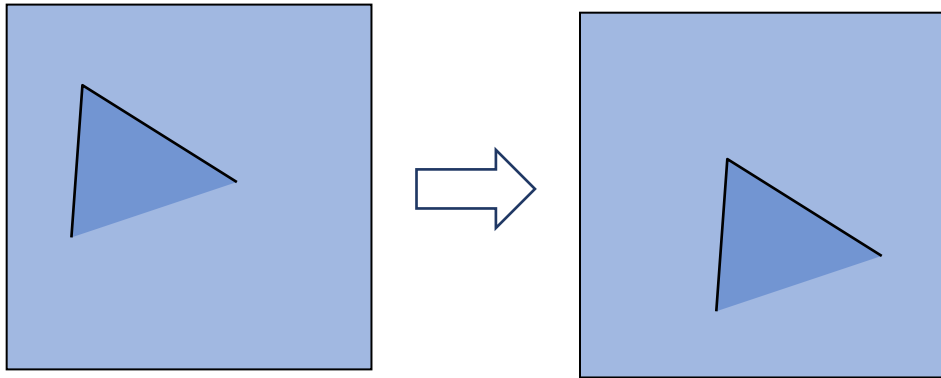
- **Preprocessing:** Convert the input image to grayscale and apply a Gaussian filter
- **Gradient:** Apply the Sobel operator to find the x and y gradient values
- **Harris value:** For each pixel p, consider a local window around it and compute the second moment matrix M and the corner strength function R
- **Threshold R and non-maximum suppression**



$$\mathbf{M} = \begin{bmatrix} \sum_{x,y \in W} I_x^2 & \sum_{x,y \in W} I_x I_y \\ \sum_{x,y \in W} I_x I_y & \sum_{x,y \in W} I_y^2 \end{bmatrix}$$

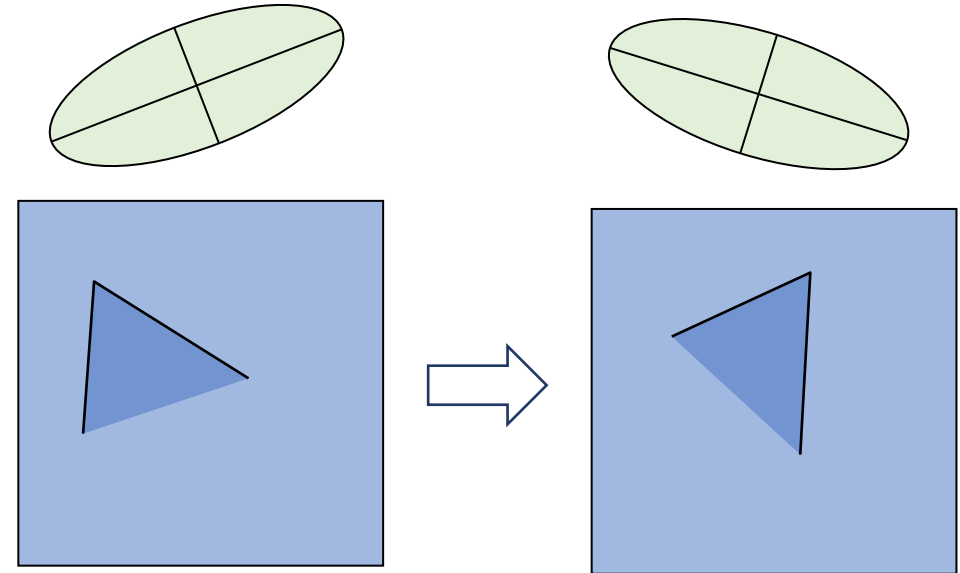
$$\begin{aligned} R &= \det(\mathbf{M}) - \alpha \operatorname{trace}(\mathbf{M})^2 \\ &= \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2 \end{aligned}$$

Properties of Corners (#1)



All done with convolutions.
Convolution is translation-equivariant.

Corners are equivariant with translation



Rotations just cause the corner orientation to change. Eigenvalues remain the same.

Corners are equivariant with rotation

Properties of Corners (#2)

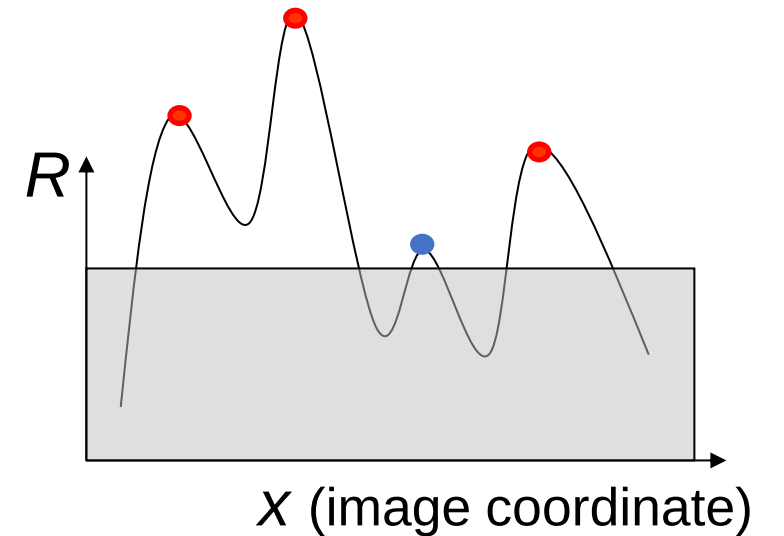
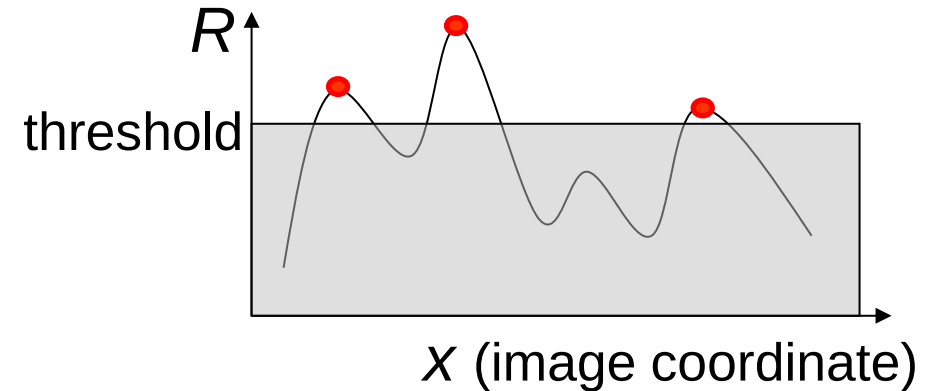
Define Affine Intensity Change as

$$I_{new} = \alpha I_{old} + \beta$$

The second moment matrix **M** only depends on derivatives, so β is irrelevant.

However, α scales the image derivatives and there are thresholds.

Therefore, **corners are partially invariant to affine intensity changes.**

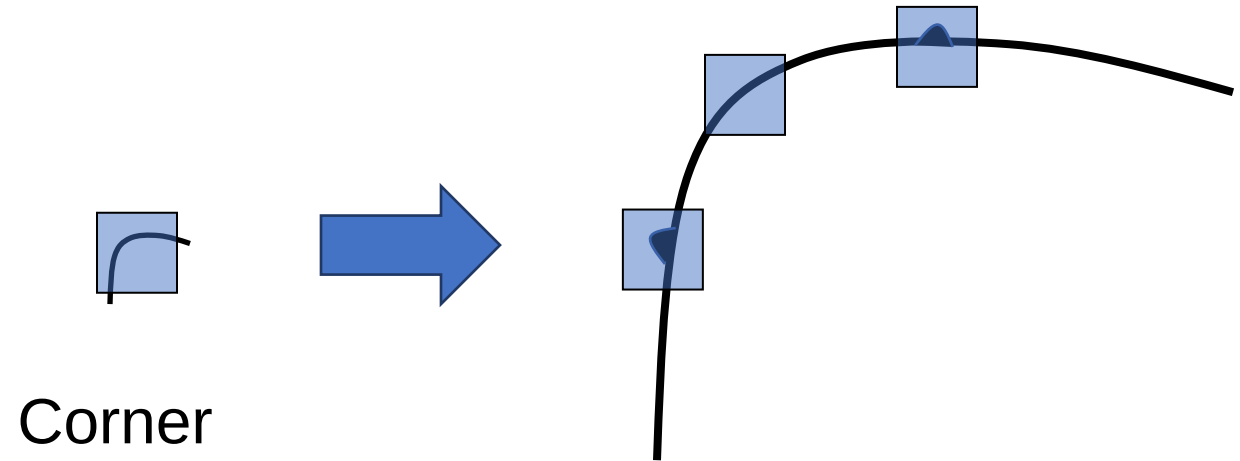


Properties of Corners (#3)

One pixel can become many pixels and vice-versa.

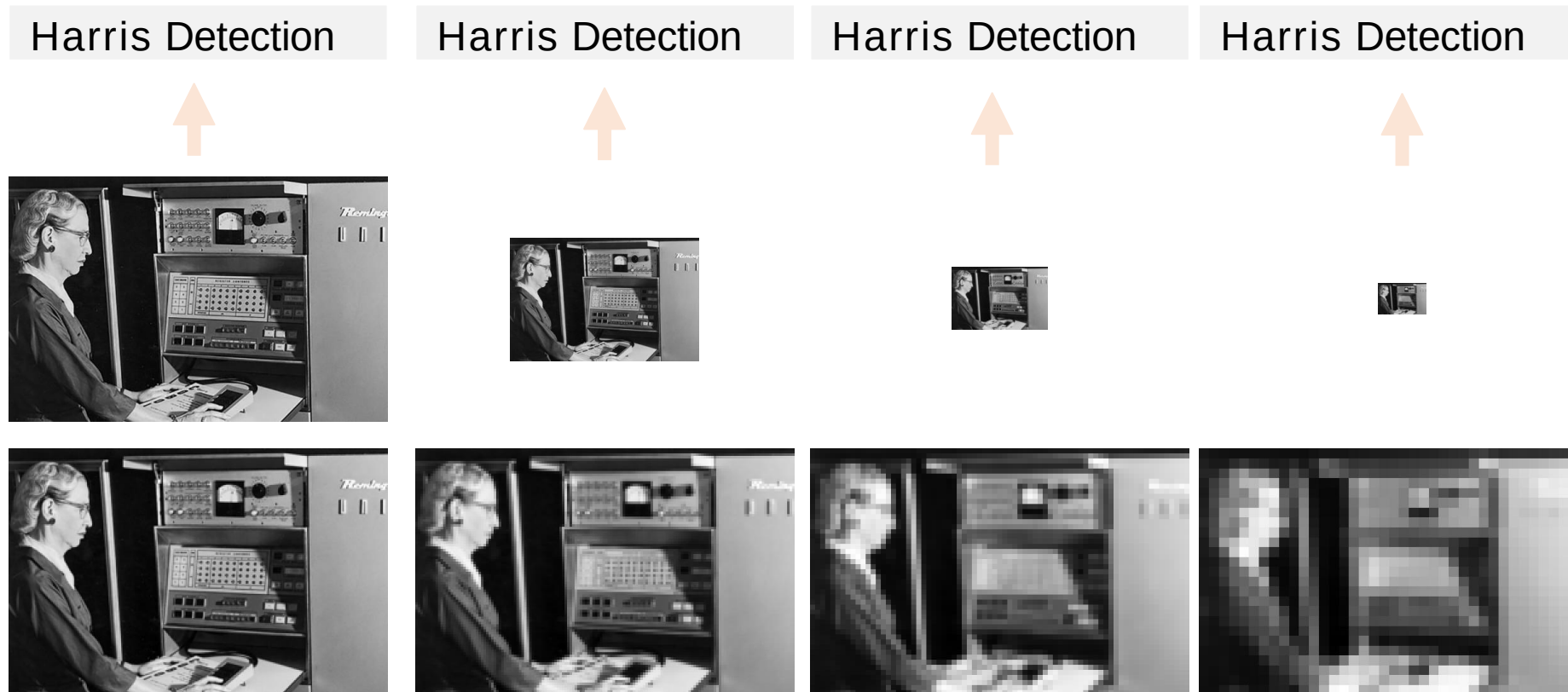
Corners are not equivariant with scaling.

Scaling can “create” or “remove” corners.

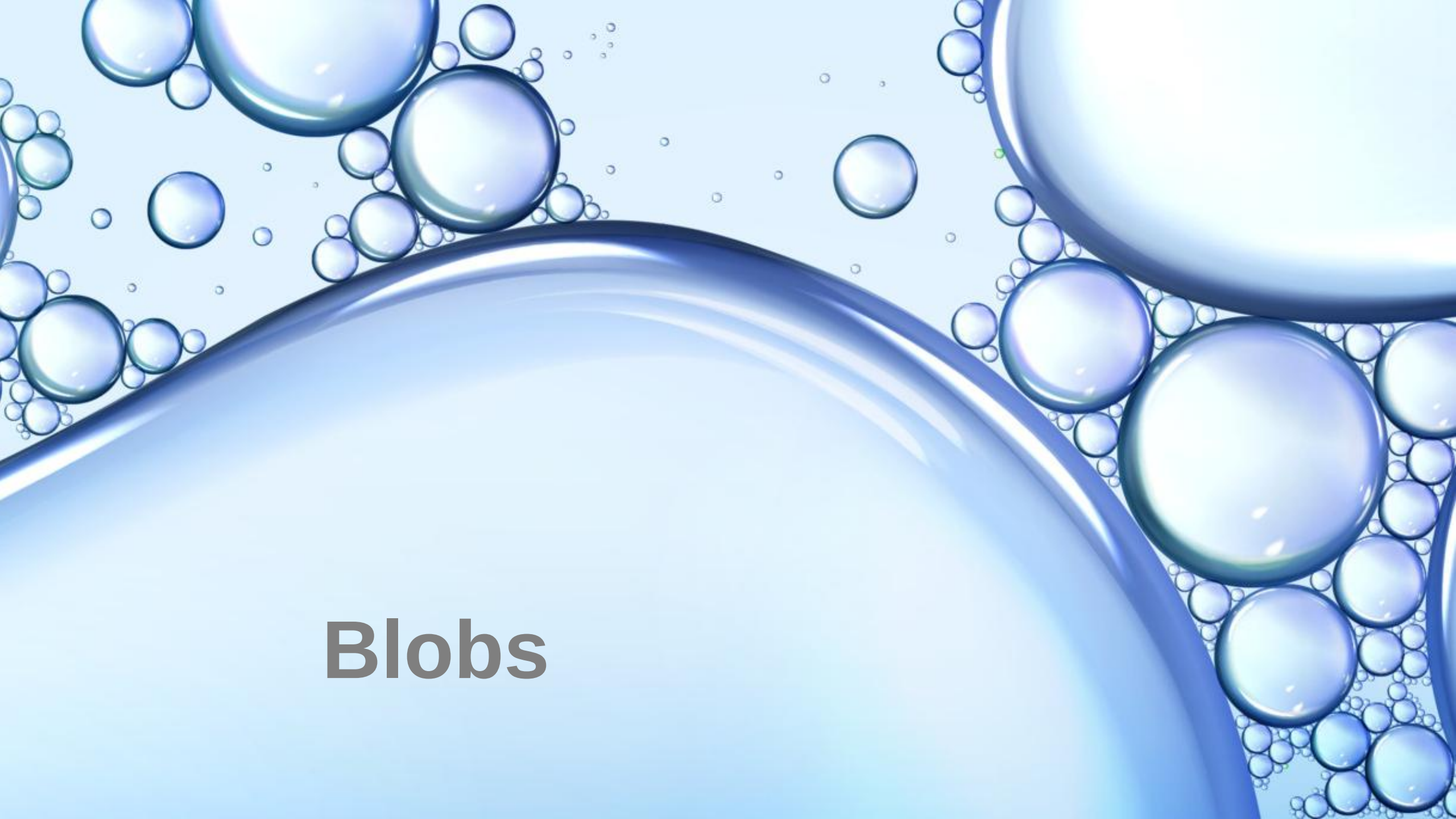


To Make Corners **Equivariant** with Scaling

- Perform Harris detection in scale space with Gaussian Pyramids



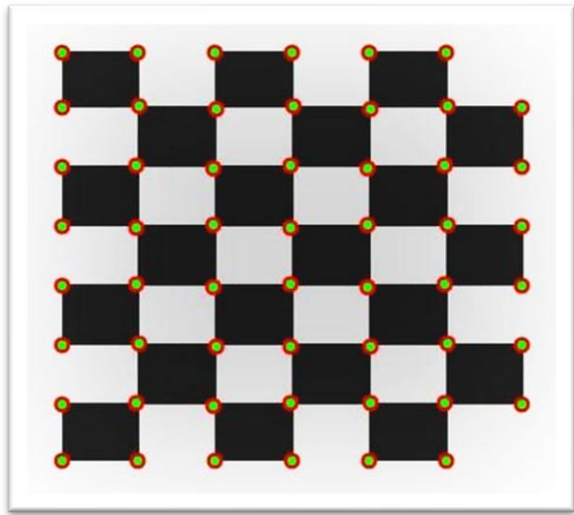
See: Multi-Image Matching using Multi-Scale Oriented Patches, Brown et al. CVPR 2005

The background is a light blue gradient. It features several large, semi-transparent blue blobs of various shapes. Interspersed among these are numerous bubbles of different sizes, some with white highlights and others with a blue tint. The overall effect is a soft, fluid, and organic composition.

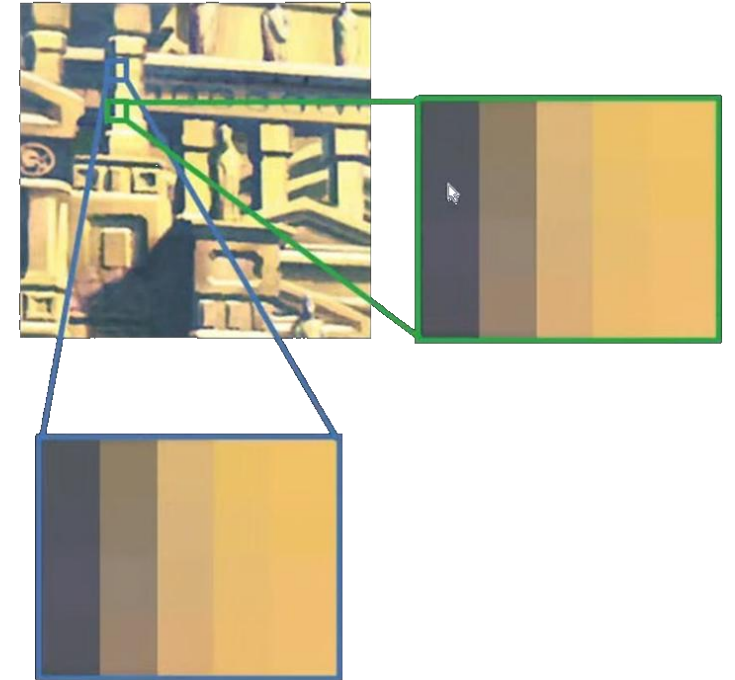
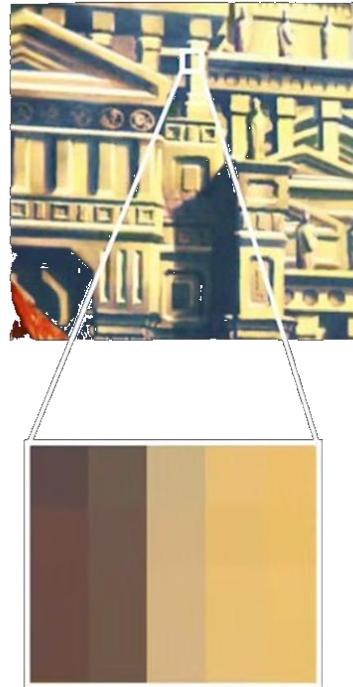
Blobs

Limitations of Corners and Edges

- Edges are hard to localize.
- Corners can be localized, but is not discriminative.



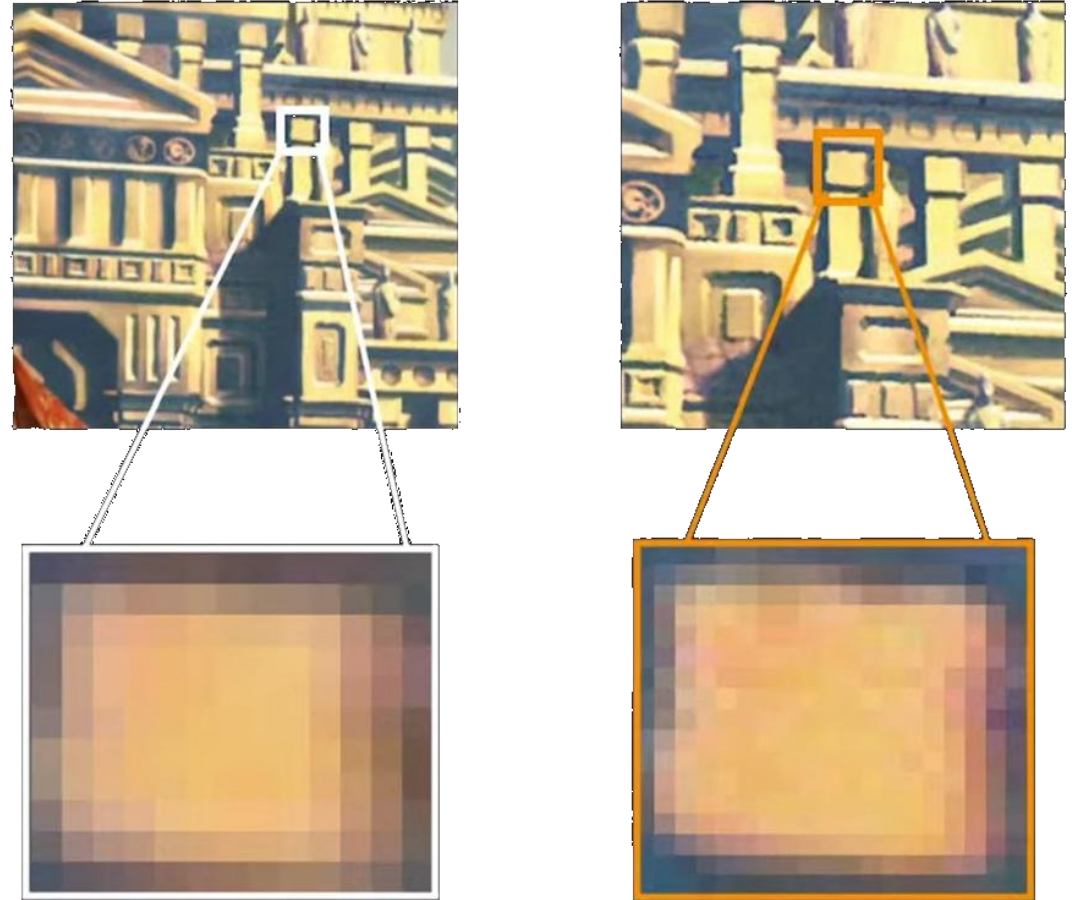
Corners



Edges

Blobs

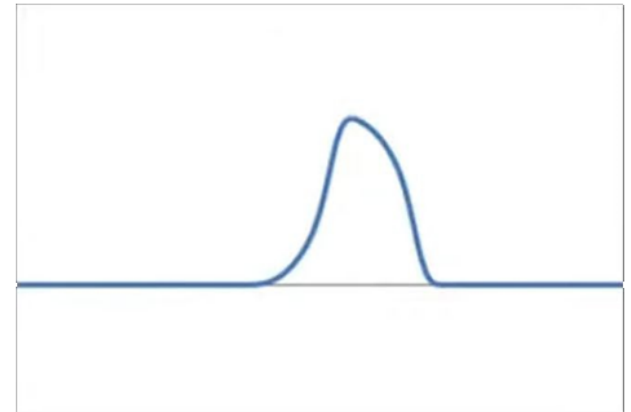
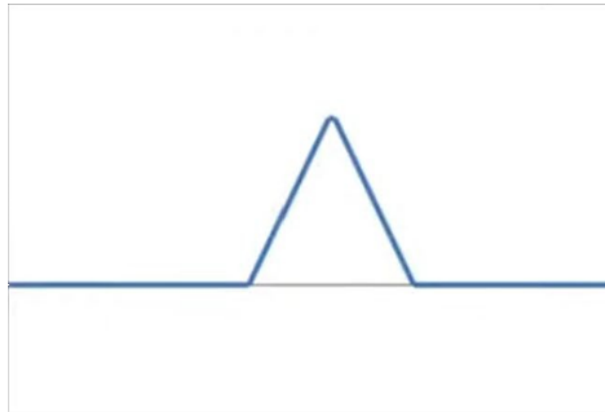
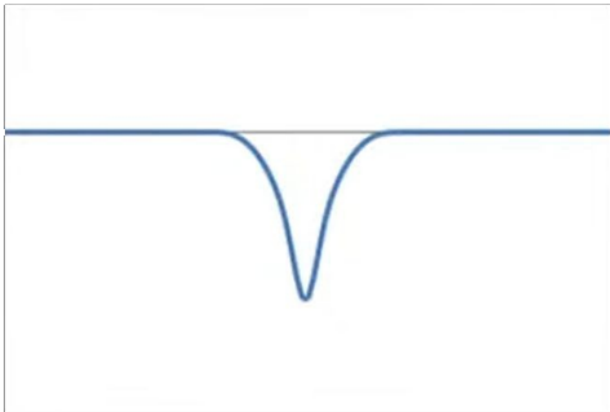
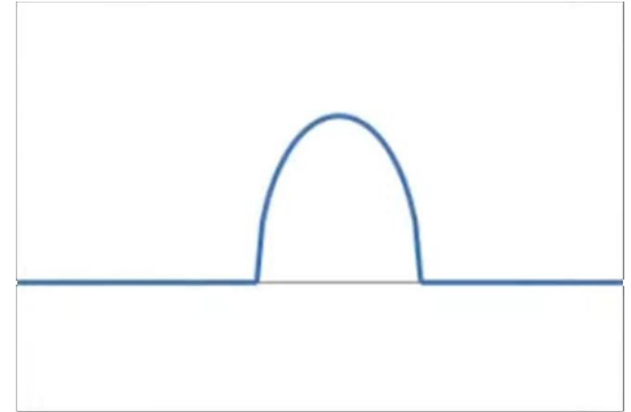
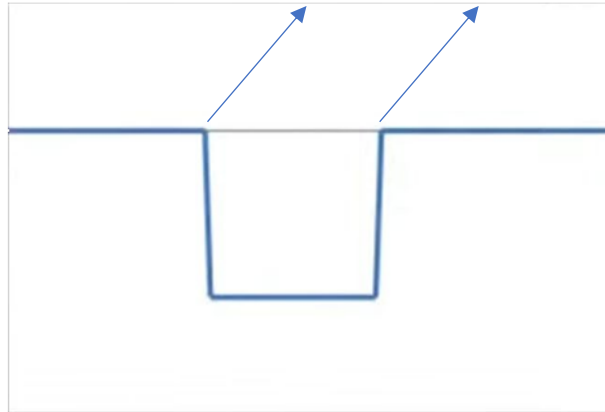
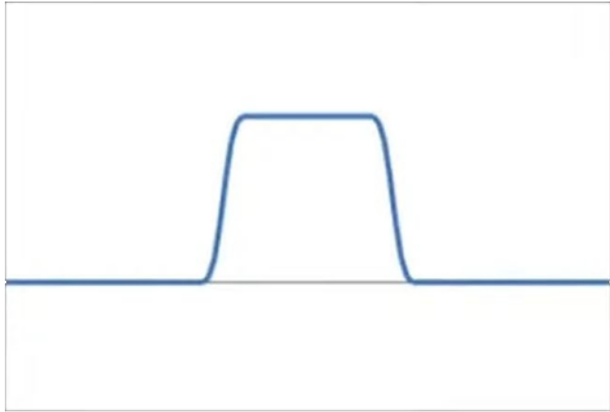
- **Blobs** are regions in a digital image that differ in properties, such as brightness or color, compared to surrounding regions.
- Blobs have fixed **positions** and **sizes**, can be **localized**, and are good interest points.
- Blob detection is often used to obtain regions of interest for further processing (the precursor of the famous SIFT feature).



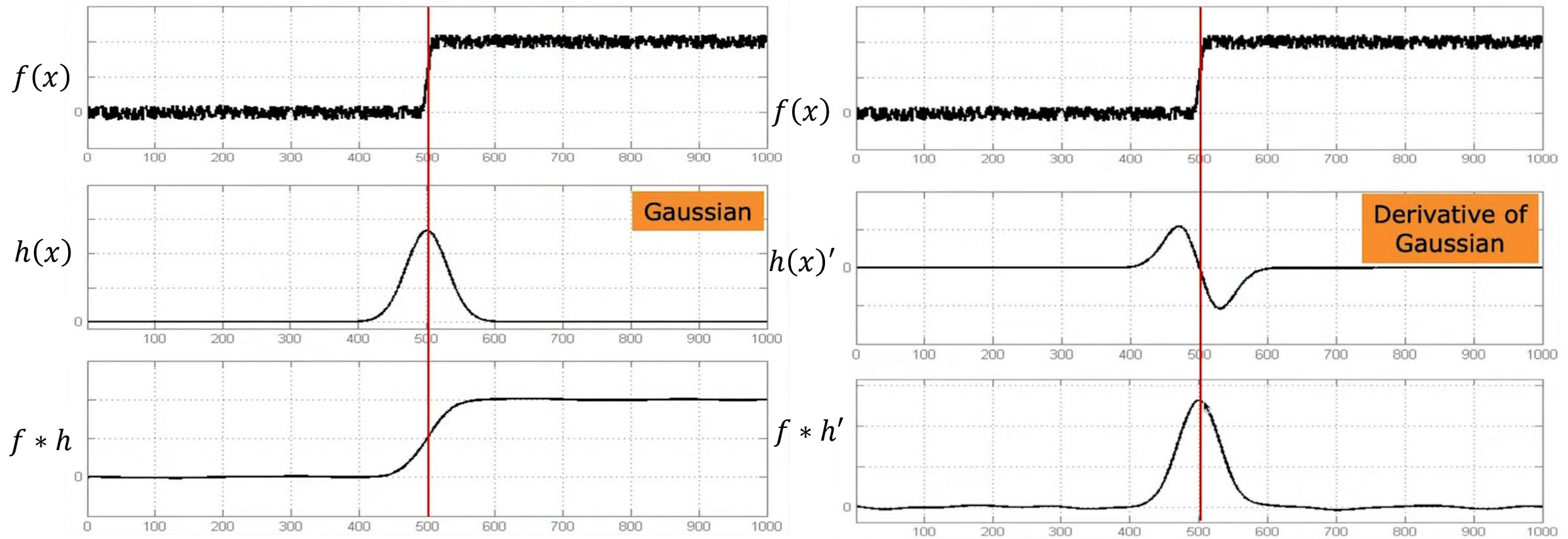
Examples of 2D Blobs

Examples of 1D Blobs

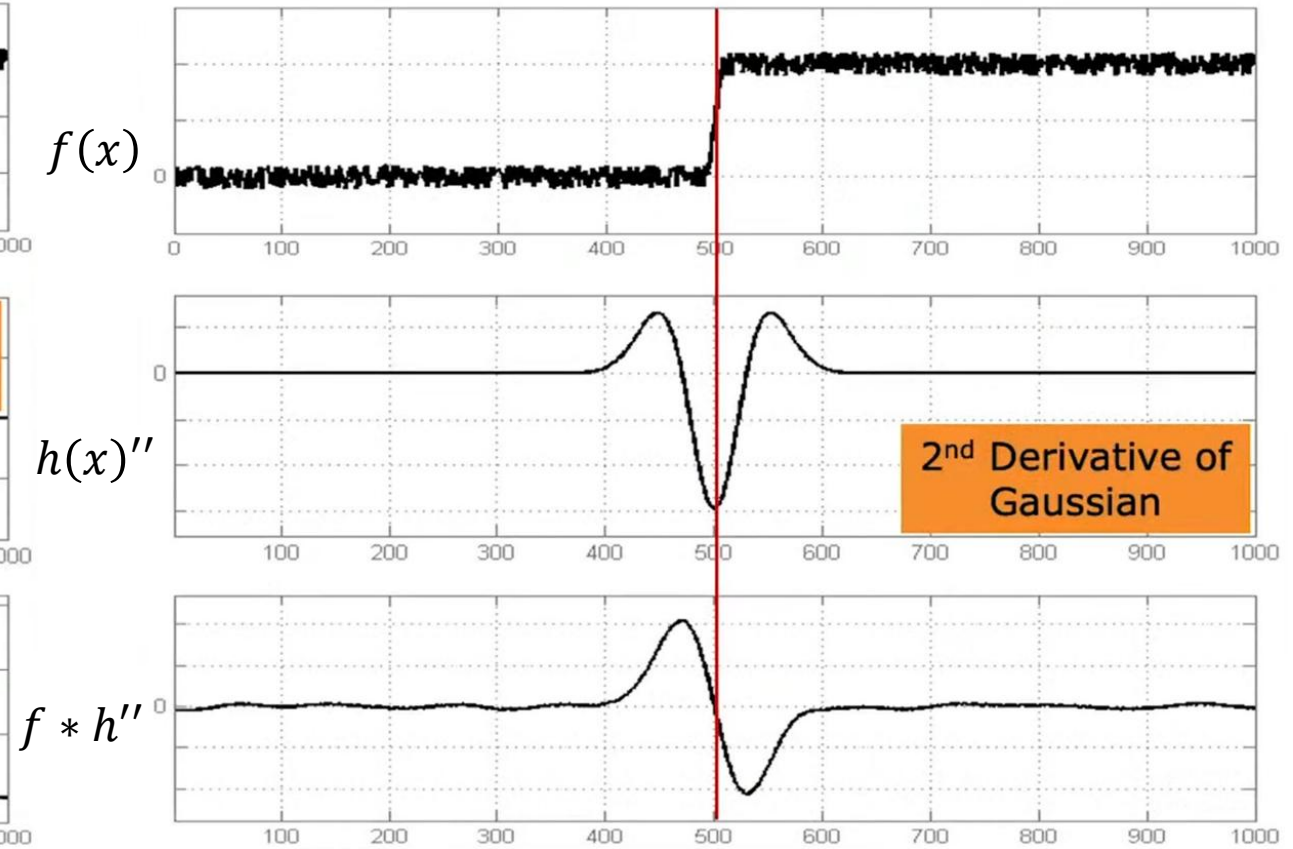
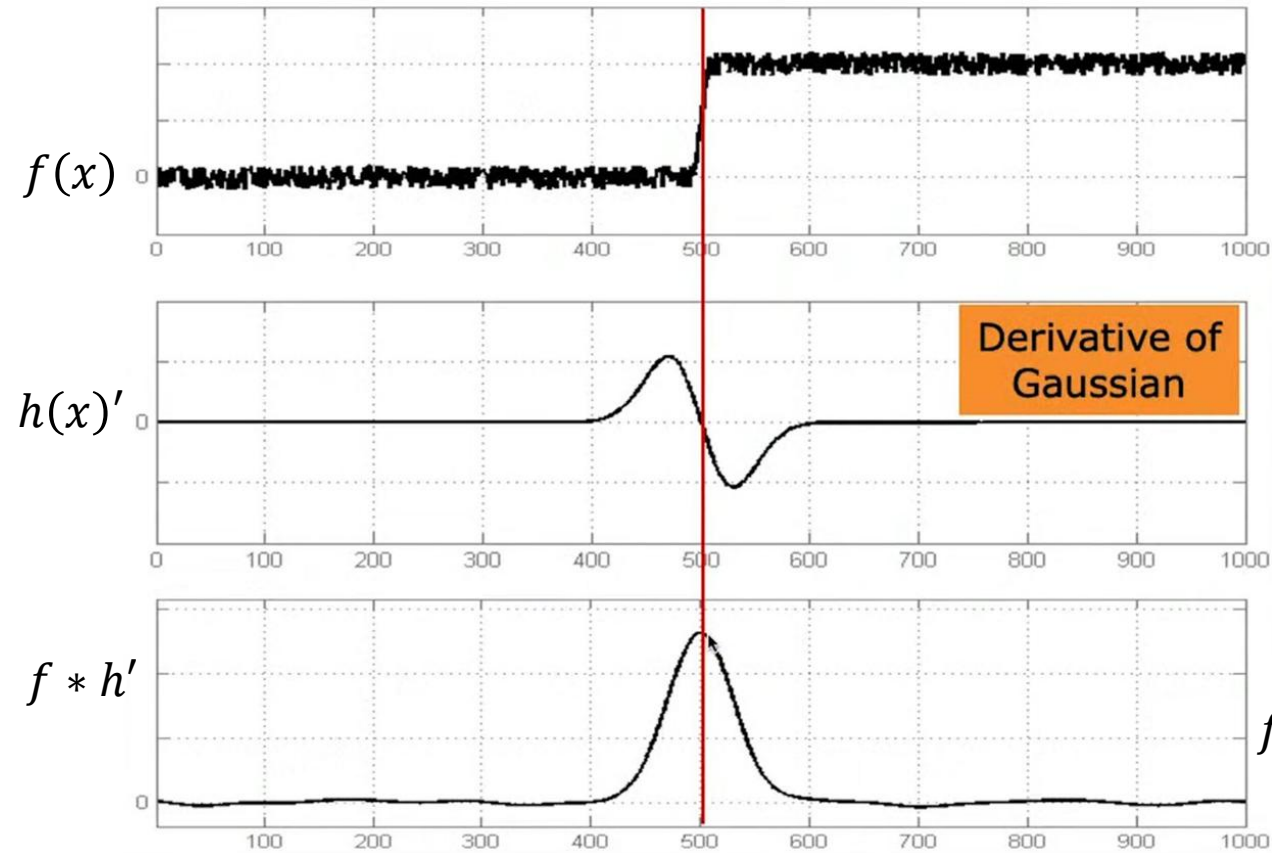
Edge detection



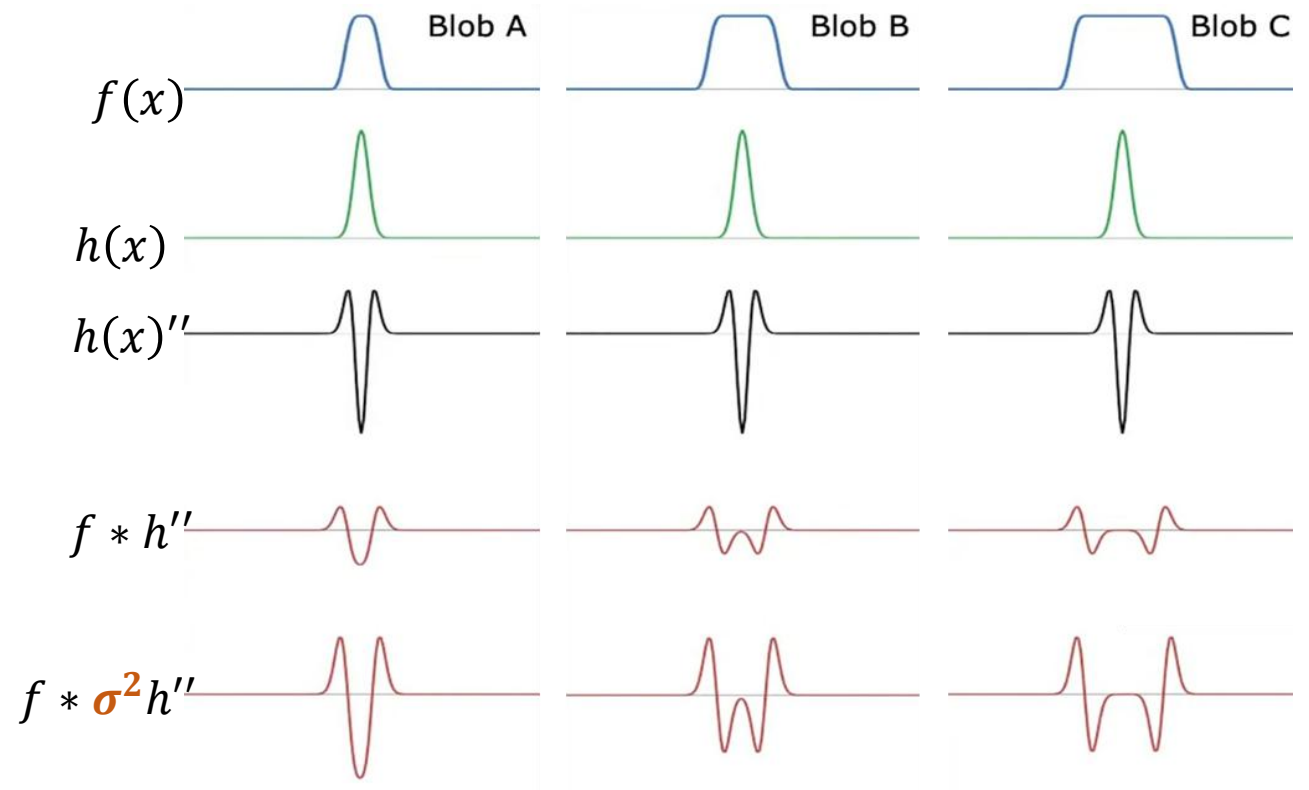
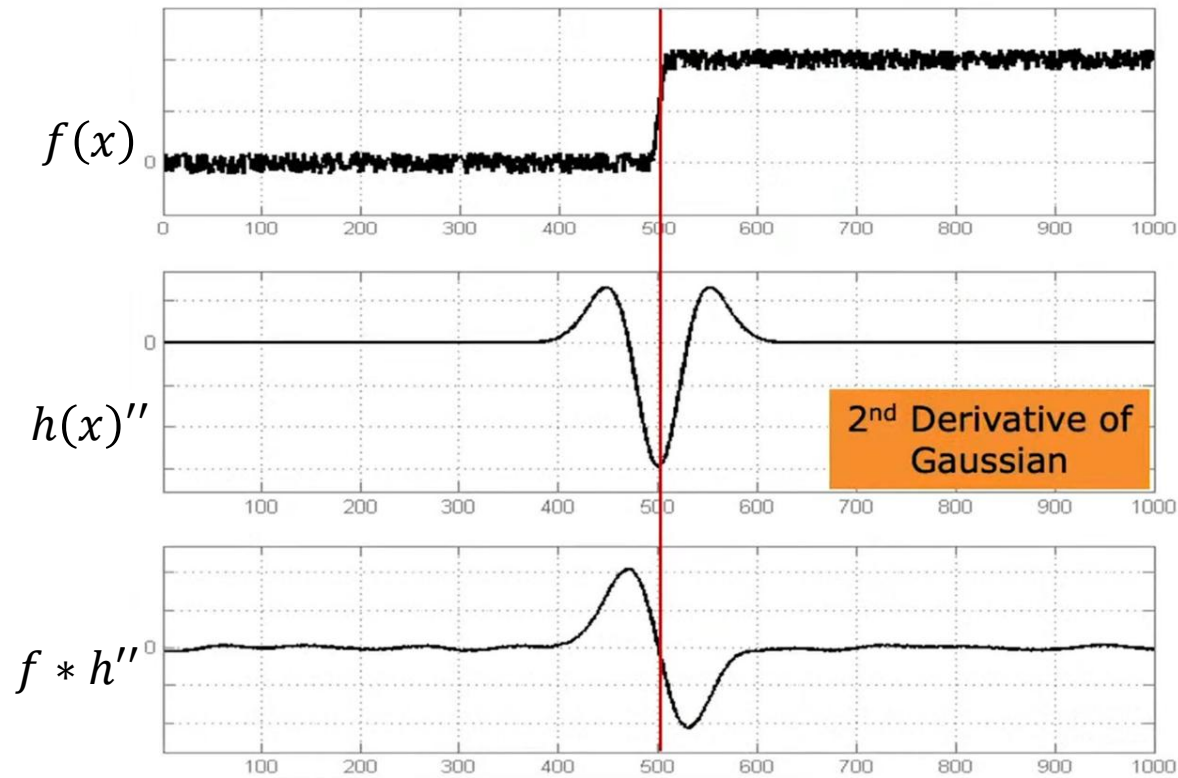
Key Ideas of Blob Detection



Key Ideas of Blob Detection



Key Ideas of Blob Detection



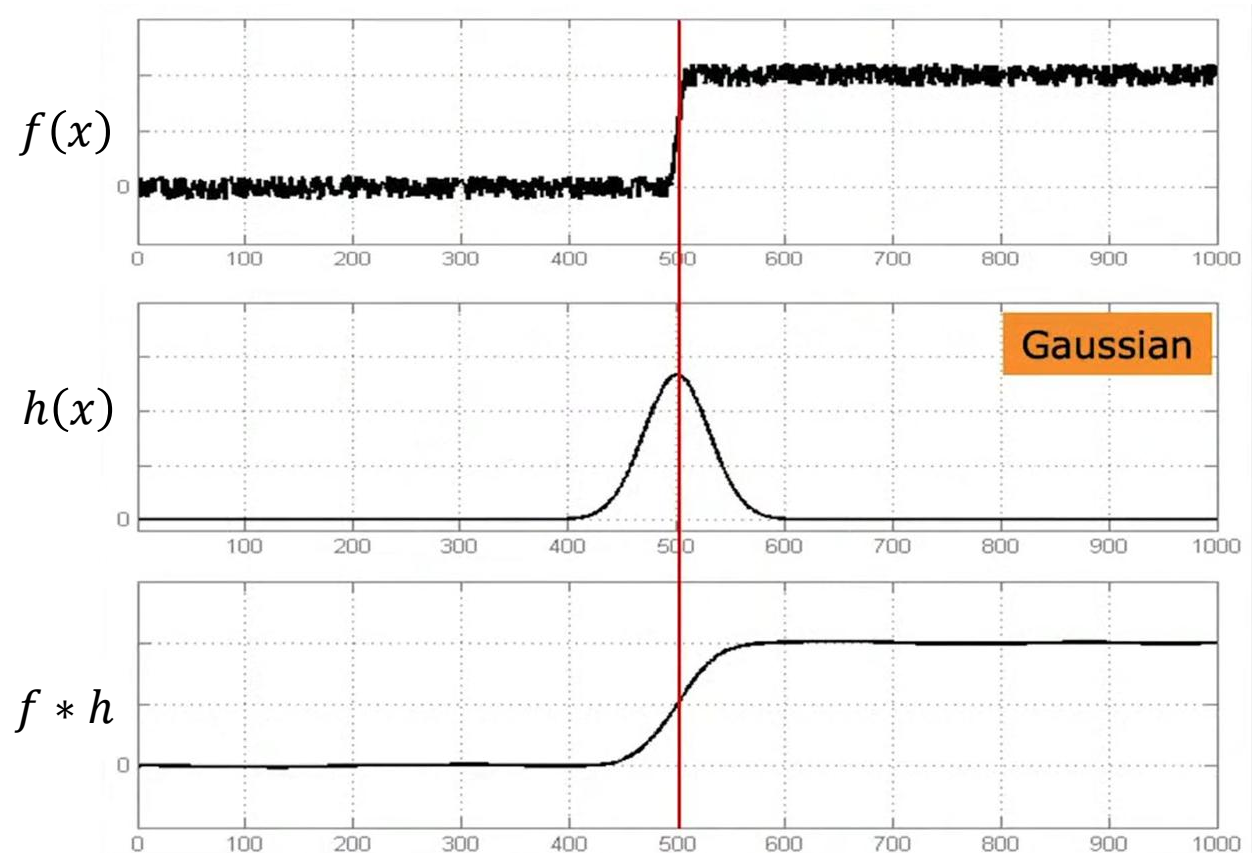
Edge Detection with Gaussian Derivatives

Filtering with convolution

$$\begin{aligned} f * h(x) &= \int f(\tau)h(x - \tau)d\tau \\ &= \int f(x - \tau)h(\tau)d\tau \end{aligned}$$

Calculate gradient for edge detection

$$\begin{aligned} \frac{d}{dx} [f * h] &= \frac{d}{dx} \int f(\tau)h(x - \tau)d\tau \\ &= \int f(\tau) \frac{d}{dx} h(x - \tau)d\tau \\ &= f * \frac{dh}{dx} = \frac{df}{dx} * h \end{aligned}$$



Edge Detection with Gaussian Derivatives

Extremum of $f * h'$ is an edge

- Set $f(x) = u(x - x_0)$ as a **step function**,

$h(x) = G_\sigma(x)$ is a Gaussian function:

$$(f * h)' = f * h' = f' * h$$

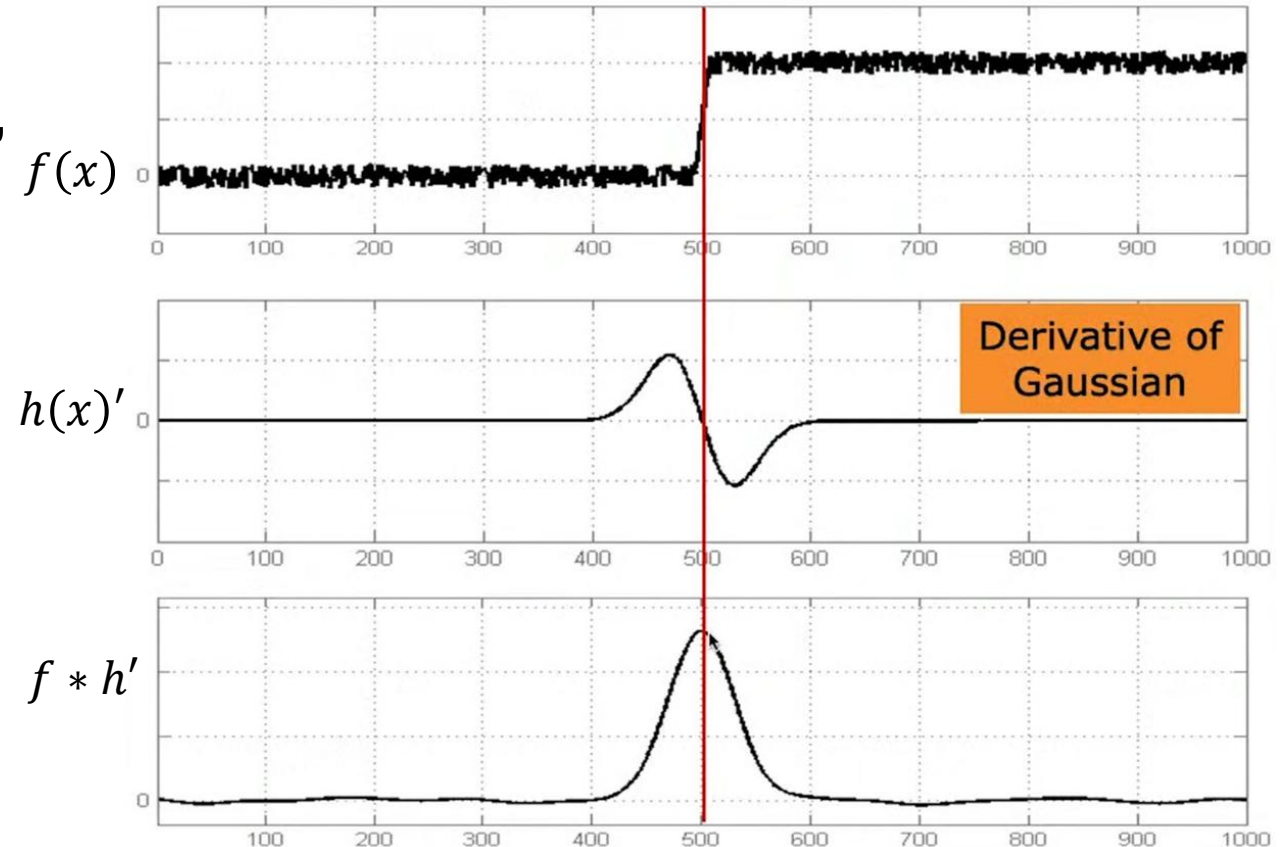
$$= G_\sigma(x) * \delta(x - x_0)$$

$$= G_\sigma(x - x_0)$$

- The edge is at the location of x_0

$$G_\sigma(0) = \frac{1}{\sqrt{2\pi}\sigma}$$

- We can use the **normalized Gaussian derivative** to make the extremum does not depend on σ , i.e., $f * \sigma G_\sigma'$



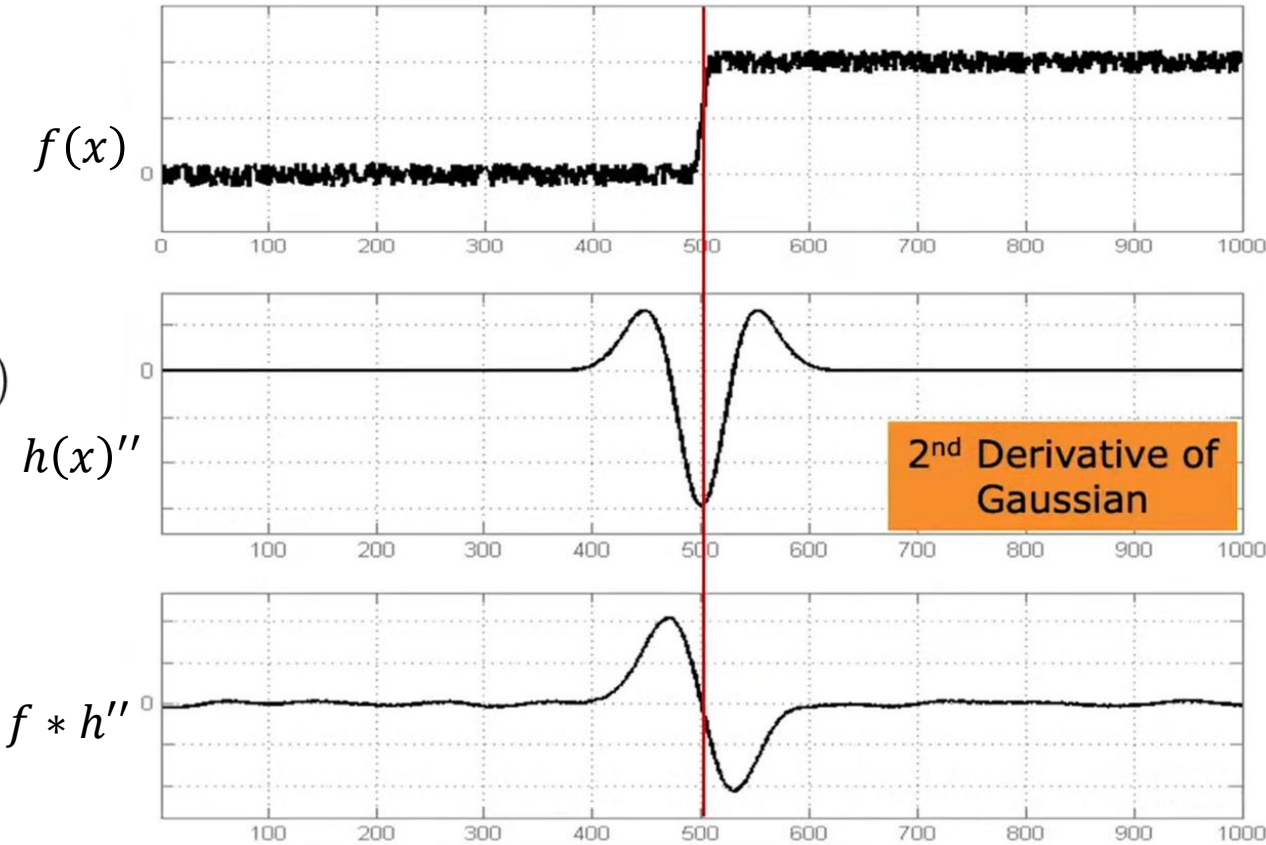
Edge Detection with the 2nd Derivative of Gaussian

Extremum of $f * h'$ is an edge

→ Zero-crossing of $f * h''$ is an edge

$$\begin{aligned} u(x - x_0) * \frac{d^2 G_\sigma(x)}{dx^2} &= \frac{d}{dx} \left(u(x - x_0) * \frac{dG_\sigma(x)}{dx} \right) \\ &= \frac{dG_\sigma(x - x_0)}{dx} \\ &= \frac{-(x - x_0)}{\sqrt{2\pi}\sigma^3} e^{-\frac{(x-x_0)^2}{2\sigma^2}} \end{aligned}$$

- The response is 0 when $x = x_0$



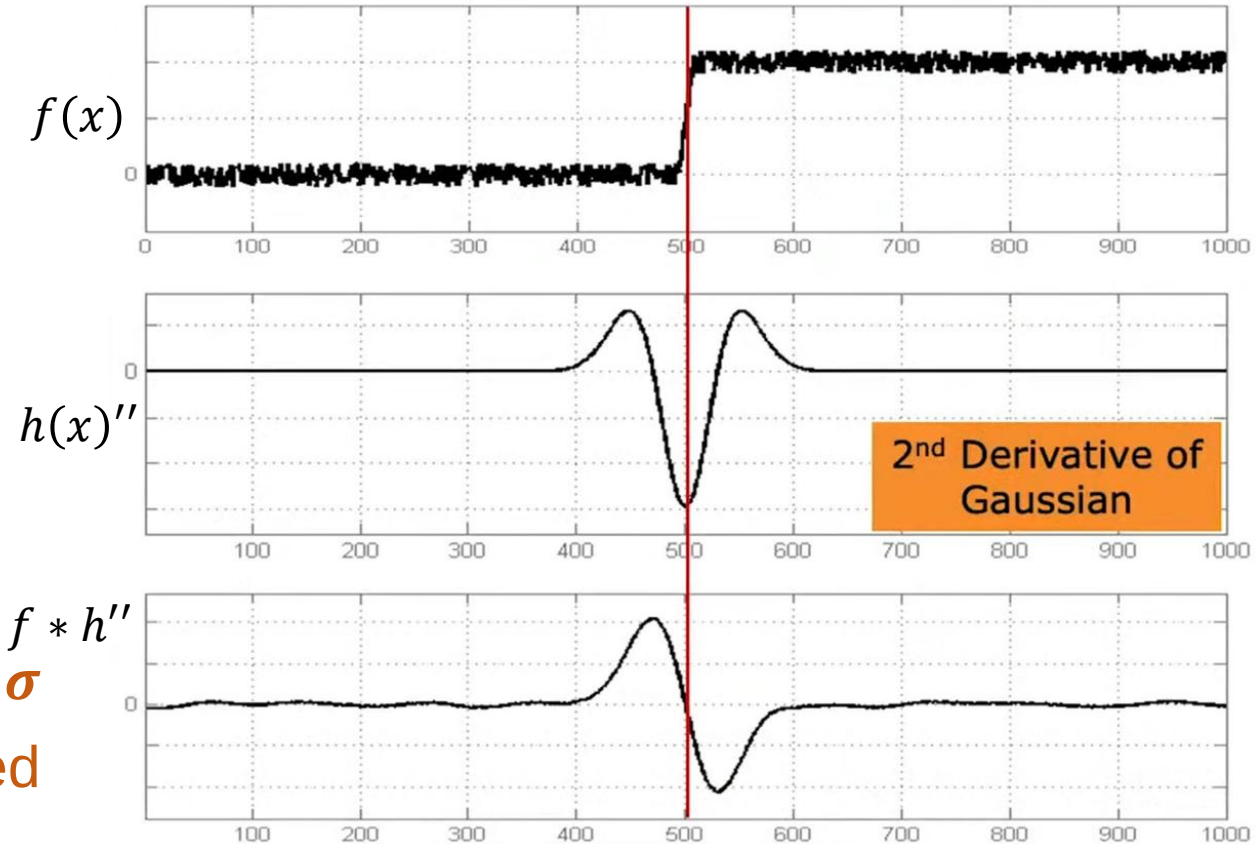
Edge Detection with the 2nd derivative of Gaussian

- Let's inspect the extremum of $f * h''$

$$u(x - x_0) * \frac{d^2 G_\sigma(x)}{dx^2} = \frac{-(x - x_0)}{\sqrt{2\pi}\sigma^3} e^{-\frac{(x-x_0)^2}{2\sigma^2}}$$

$$\begin{aligned} \frac{d}{dx} \left(u(x - x_0) * \frac{d^2 G_\sigma(x)}{dx^2} \right) \\ = -\left(1 - \frac{(x - x_0)^2}{\sigma^2}\right) \frac{1}{\sqrt{2\pi}\sigma^3} e^{-\frac{(x-x_0)^2}{2\sigma^2}} \end{aligned}$$

- The extremums of $f * h''$ are at $x = x_0 \pm \sigma$ with values $\pm \frac{1}{\sqrt{2\pi}\sigma^2} e^{-\frac{1}{2}}$. Use the normalized 2nd derivative of Gaussian: $f * \sigma^2 G_\sigma''$.



Blob Detection with the 2nd Derivative of Gaussian

- Filter with the 2nd Derivative of Gaussian

$$u(x - x_0) * \frac{d^2 G_\sigma(x)}{dx^2} = \frac{-(x - x_0)}{\sqrt{2\pi}\sigma^3} e^{-\frac{(x-x_0)^2}{2\sigma^2}}$$

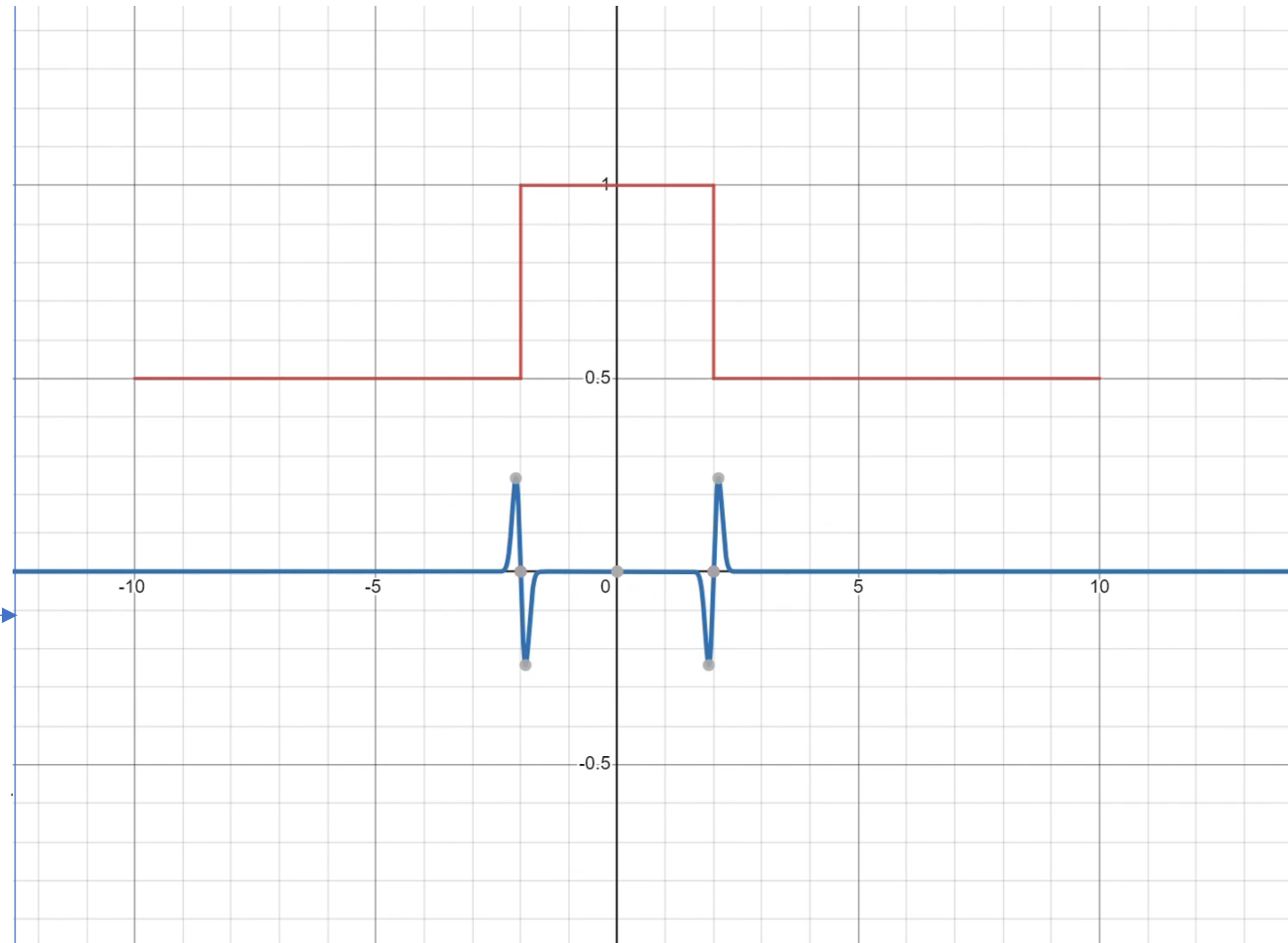
- Define a 1D Blob as

$$f(x) = u(x - x_0) - u(x - x_1)$$

- Filter the 1D Blob

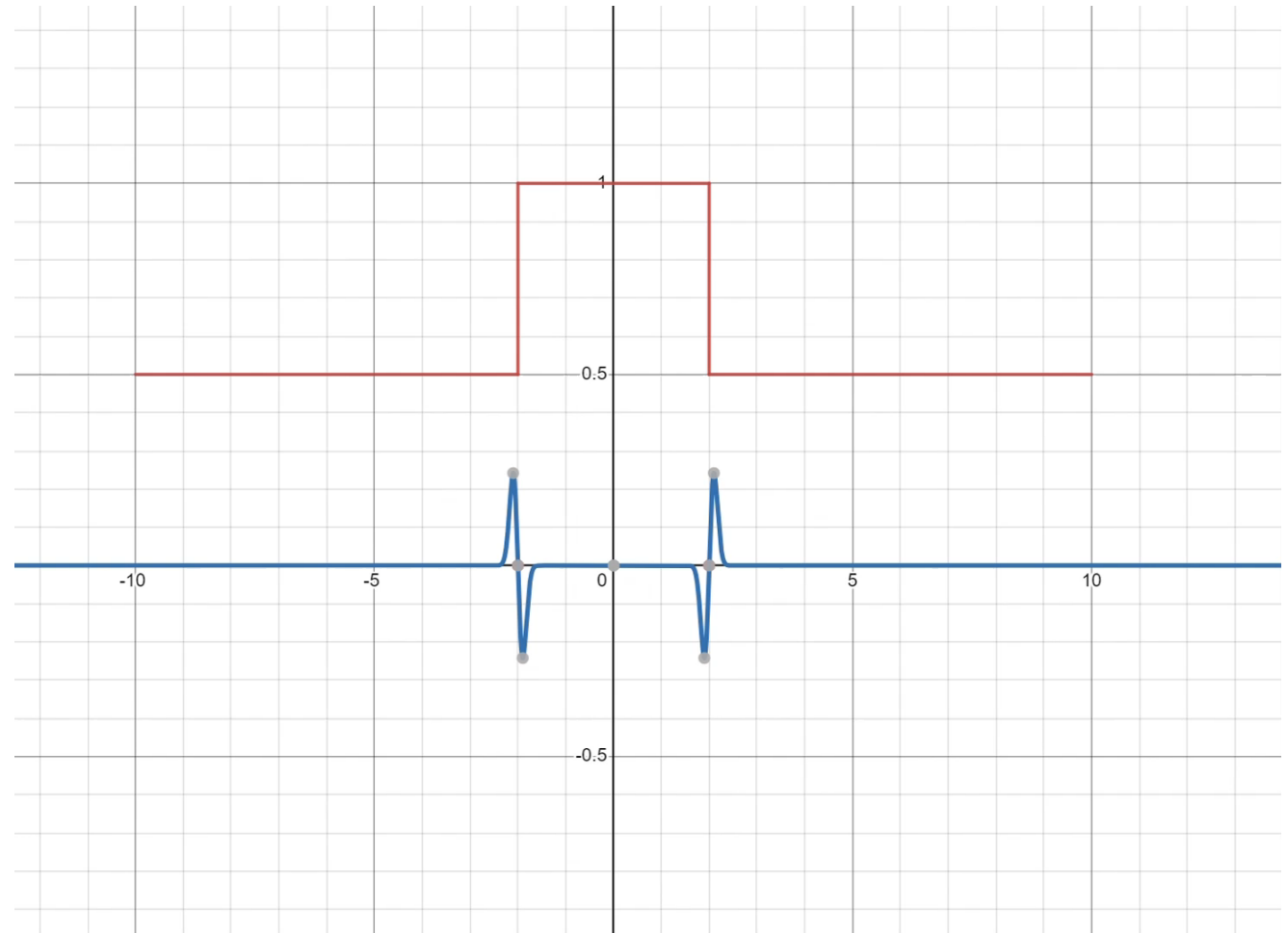
$$f(x) * \sigma^2 \frac{d^2 G_\sigma(x)}{dx^2} = -\frac{(x - x_0)}{\sqrt{2\pi}\sigma} e^{-\frac{(x-x_0)^2}{2\sigma^2}} + \frac{(x - x_1)}{\sqrt{2\pi}\sigma} e^{-\frac{(x-x_1)^2}{2\sigma^2}}$$

- When $\sigma = \frac{x_1 - x_0}{2}$, the extremums coincide at $x = x_0 + \sigma = x_1 - \sigma = \frac{x_1 + x_0}{2}$



Summary of 1D Blob Detection

- Filter the function $f(x)$ with **multiscale normalized** the 2nd derivative of Gaussians, the result is denoted as $L(x, \sigma)$.
- The blobs are detected by the extremums of $L(x, \sigma)$:
 $(\hat{x}, \hat{\sigma}) = \arg \text{MinMax } L(x, \sigma)$
- Here $\hat{x}$ and $\hat{\sigma}$ are the **position** and the **characteristic size** of the blob.
- **Characteristic size $\propto$ Blob size**



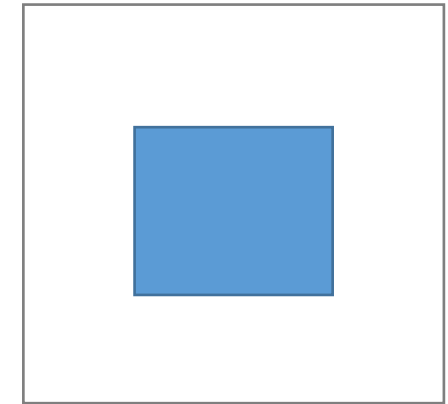
2D Blob Detection: Laplacian of Gaussian

- Intuitively, we can detect blobs in the x direction with the 2nd derivative of Gaussian, $G''(x; \sigma)$, then in the y direction with $G''(y; \sigma)$, and **sum** the response.

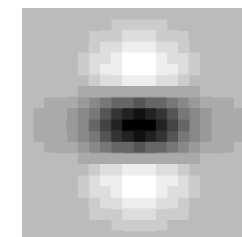
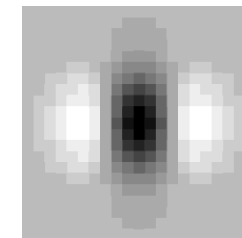
$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$\nabla^2 G(x, y; \sigma) = \left(\frac{x^2 + y^2}{\sigma^4} - \frac{2}{\sigma^2} \right) G(x, y; \sigma)$$

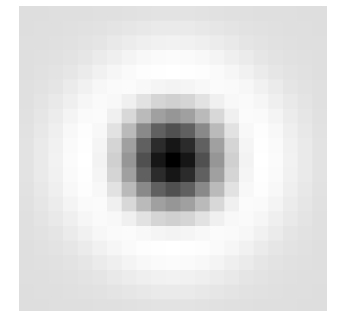
- 2D blobs can be detected with multiscale Normalized Laplacian of Gaussian (**NLoG**: $\sigma^2 \nabla^2 G(x, y; \sigma)$), i.e., **filter images with different σ values, and extract extremums.**



2D Blob



+



Laplacian of Gaussian

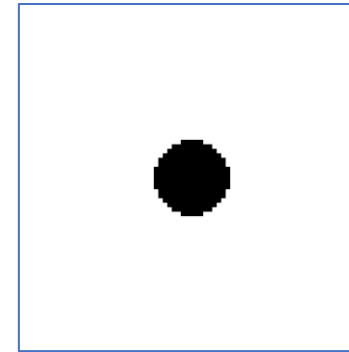
2D Blob Detection: Scale Selection

Given a binary circle and NLoG filter of scale σ , we can compute the response as a function of the scale $I(x, y; \sigma)$.

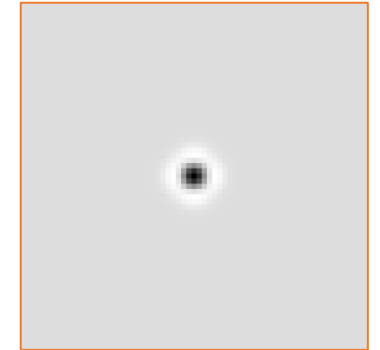
```
for y in range(i-1, i+1+1):  
    for x in range(j-1, j+1+1):  
        for sigma in range(k-1, k+1+1):  
            # find c to ensure the following  
            # line to be true  
            I[y, x, c] < I[i, j, sigma]
```

R is the response of NLoG on the circle center. 

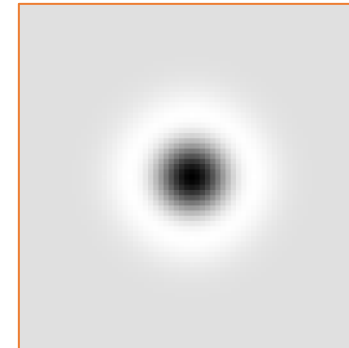
Input
Radius: 8



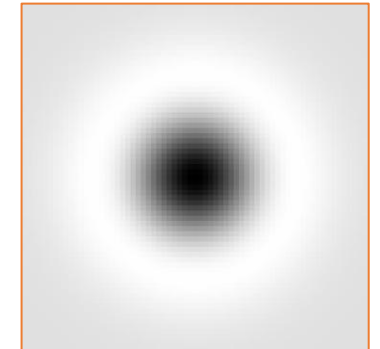
$\sigma = 2$
R: 0.02



$\sigma = 6$
R: 2.9



$\sigma = 10$
R: 1.8



Summary of 2D Blob Detection

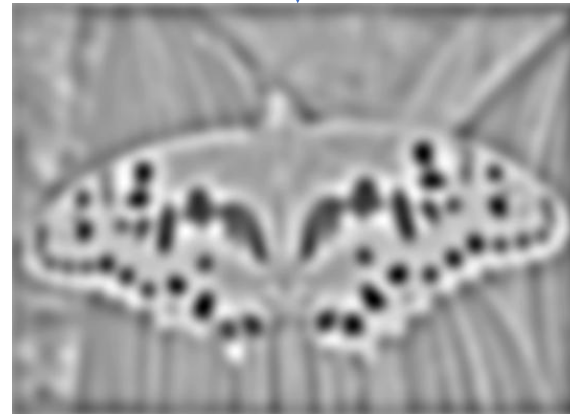
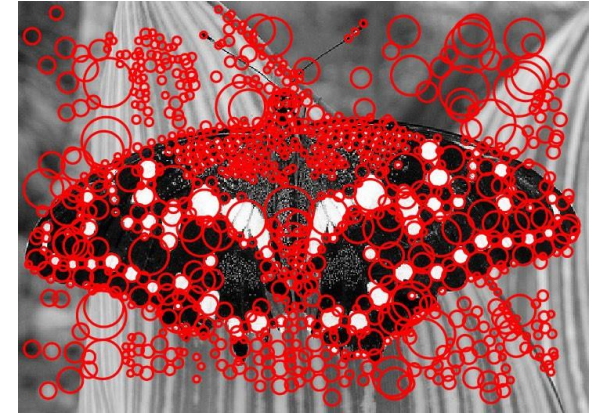
- Filter the image $I(x, y)$ NLOG with different scales

$$L(x, y; \sigma) = \sigma^2 \nabla G_\sigma(x, y) * I(x, y)$$

- The blobs are detected by the extremums of $L(x, y, \sigma)$:

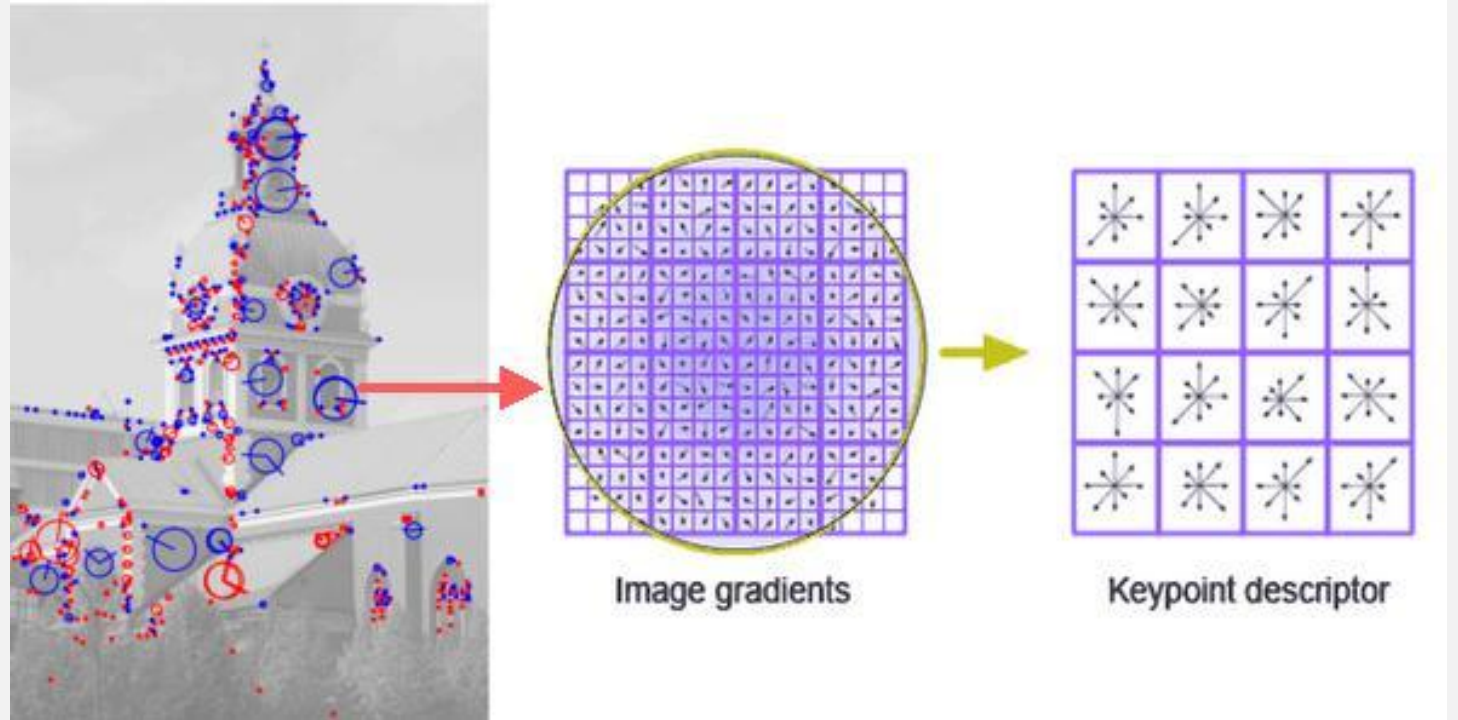
$$(\hat{x}, \hat{y}, \hat{\sigma}) = \arg \text{MinMax} L(x, y; \sigma)$$

- Here $(\hat{x}, \hat{y})$ and $\hat{\sigma}$ are the **position** and the **characteristic size** of the blob.



SIFT Feature

Distinctive Image Features
from Scale-Invariant Keypoints

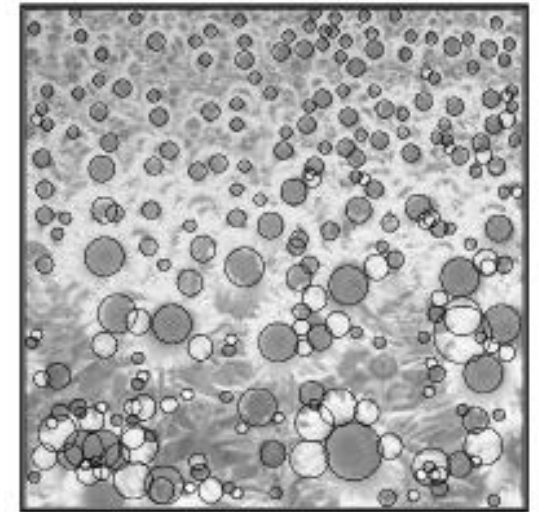


Blobs as Scale-Invariant Keypoints

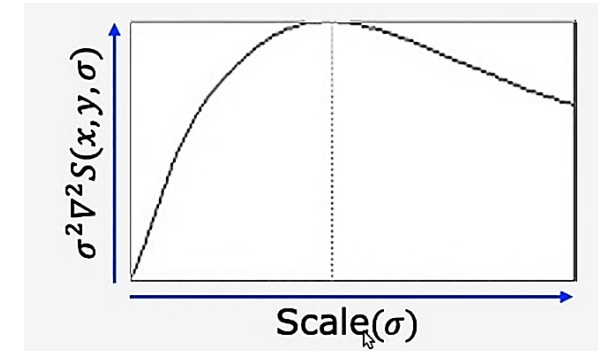
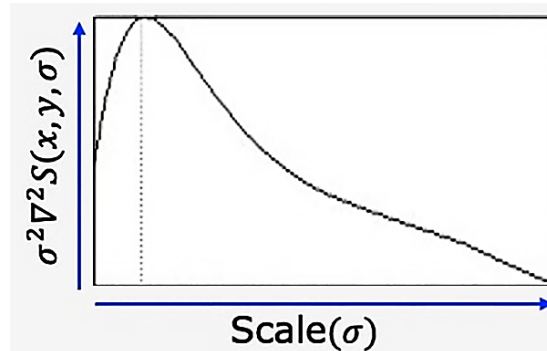
- Blobs are detected by extracting extremums of multiscale NLoG
- If a blob is rescaled, it can still be detected in a **shifted** filtering output of NLoG; thus **blobs are scale-equivariant**
- After **scale normalization**, we make the blob **scale-invariant**
- Blobs are also robust to rotation, occlusion, clutter, or noise



Input Image



Blobs



Fast NLoG Approximation: DoG

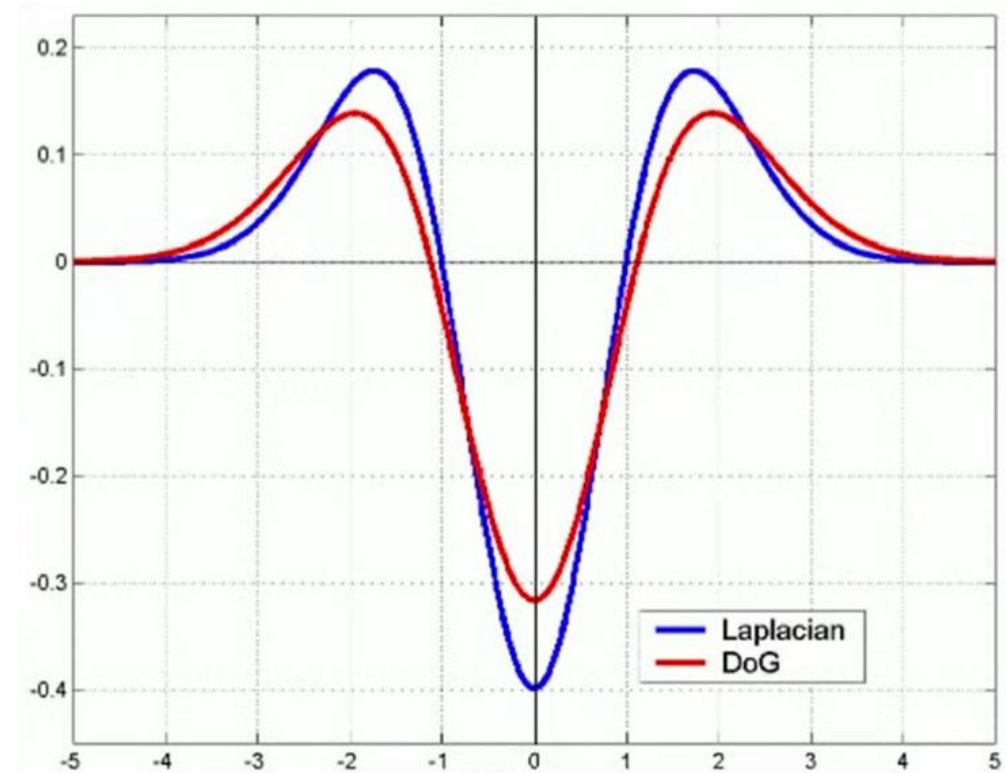
- Difference of Gaussian (DoG) is an approximation of **normalized** Laplacian of Gaussian (NLoG)

$$G(x, y; \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\nabla^2 G(x, y; \sigma) = \left(\frac{x^2 + y^2}{\sigma^4} - \frac{2}{\sigma^2} \right) G(x, y; \sigma) = \frac{\partial G}{\partial \sigma} \frac{1}{\sigma}$$

$$\sigma \nabla^2 G = \frac{\partial G}{\partial \sigma} \approx \frac{G(x, y, k\sigma) - G(x, y, \sigma)}{k\sigma - \sigma}$$

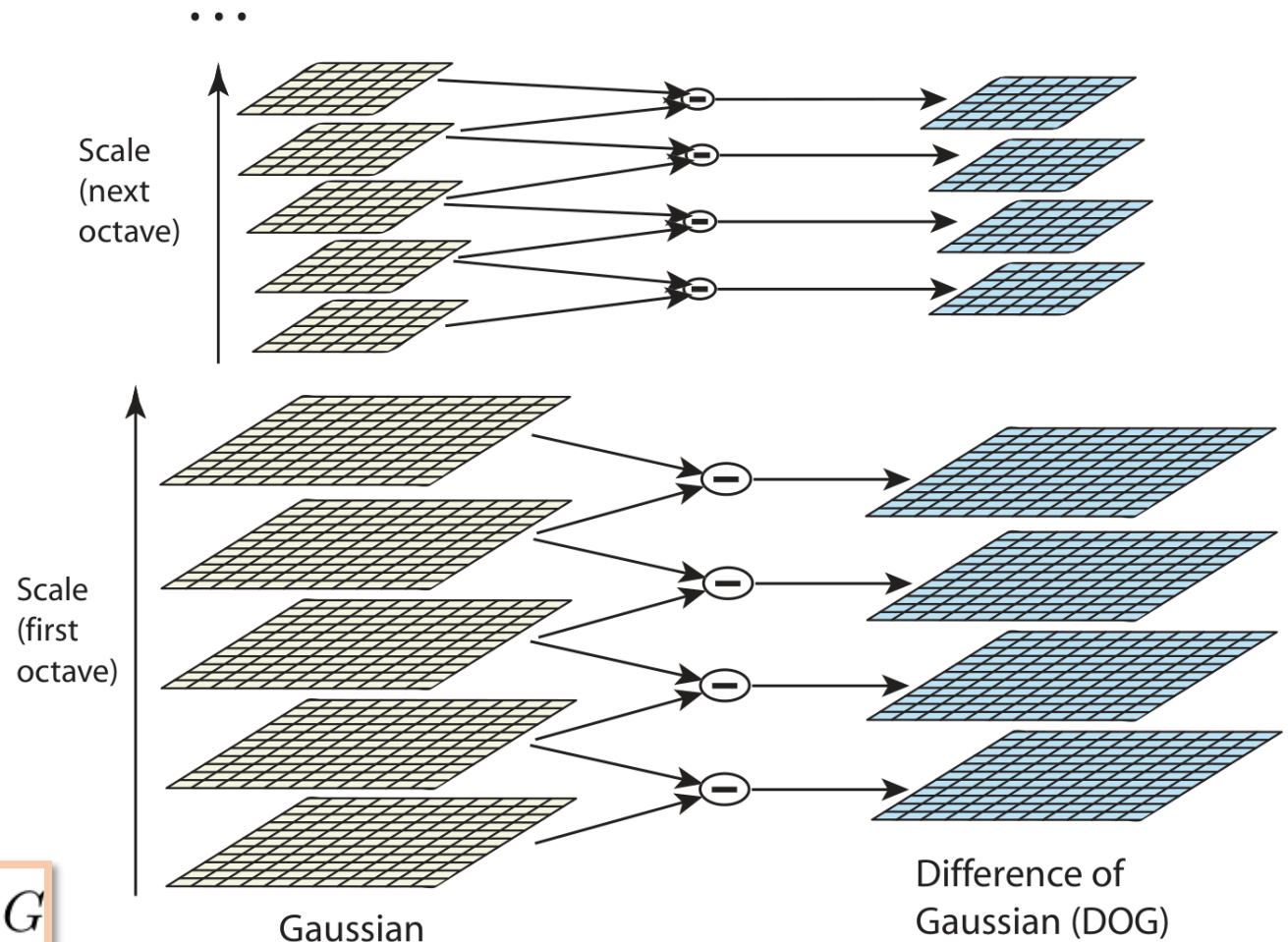
$$G(x, y, k\sigma) - G(x, y, \sigma) \approx (k - 1)\sigma^2 \nabla^2 G$$



Blob Detection with the Image Pyramid

- Build a Gaussian pyramid with standard deviations ($k = \sqrt{2}$)
 $\{1, \sigma, k\sigma, k^2\sigma, \dots, k^n\sigma\}$
- The first octave
Gaussian: $\{\sigma, \sqrt{2}\sigma, \underline{2\sigma}, 2\sqrt{2}\sigma, 4\sigma\}$
 $\rightarrow DoG: \{\sqrt{2}\sigma, 2\sigma, \underline{2\sqrt{2}\sigma}, 4\sigma\}$
- The second octave (**downsample**)
Gaussian: $\{\underline{2\sigma}, 2\sqrt{2}\sigma, 4\sigma, 4\sqrt{2}\sigma, 8\sigma\}$
 $\rightarrow DoG: \{2\sqrt{2}\sigma, 4\sigma, \underline{4\sqrt{2}\sigma}, 8\sigma\}$

$$G(x, y, k\sigma) - G(x, y, \sigma) \approx (k - 1)\sigma^2 \nabla^2 G$$



Math: Associative Law of Convolution

$$[f_1(t) * f_2(t)] * f_3(t)$$

$$= \int_{-\infty}^{\infty} \left[\int_{-\infty}^{\infty} f_1(\lambda) f_2(\tau - \lambda) d\lambda \right] f_3(t - \tau) d\tau$$

$$= \int_{-\infty}^{\infty} f_1(\lambda) \left[\int_{-\infty}^{\infty} f_2(\tau - \lambda) f_3(t - \tau) d\tau \right] d\lambda$$

$$= \int_{-\infty}^{\infty} f_1(\lambda) \left[\int_{-\infty}^{\infty} f_2(\tau) f_3(t - \tau - \lambda) d\tau \right] d\lambda$$

$$= \int_{-\infty}^{\infty} f_1(\lambda) [f_2(t - \lambda) * f_3(t - \lambda)] d\lambda$$

$$= f_1(t) * [f_2(t) * f_3(t)]$$

$$f_X(x) = \mathcal{N}(x; \mu_X, \sigma_X^2) = \frac{1}{\sqrt{2\pi}\sigma_X} e^{-(x-\mu_X)^2/(2\sigma_X^2)}$$

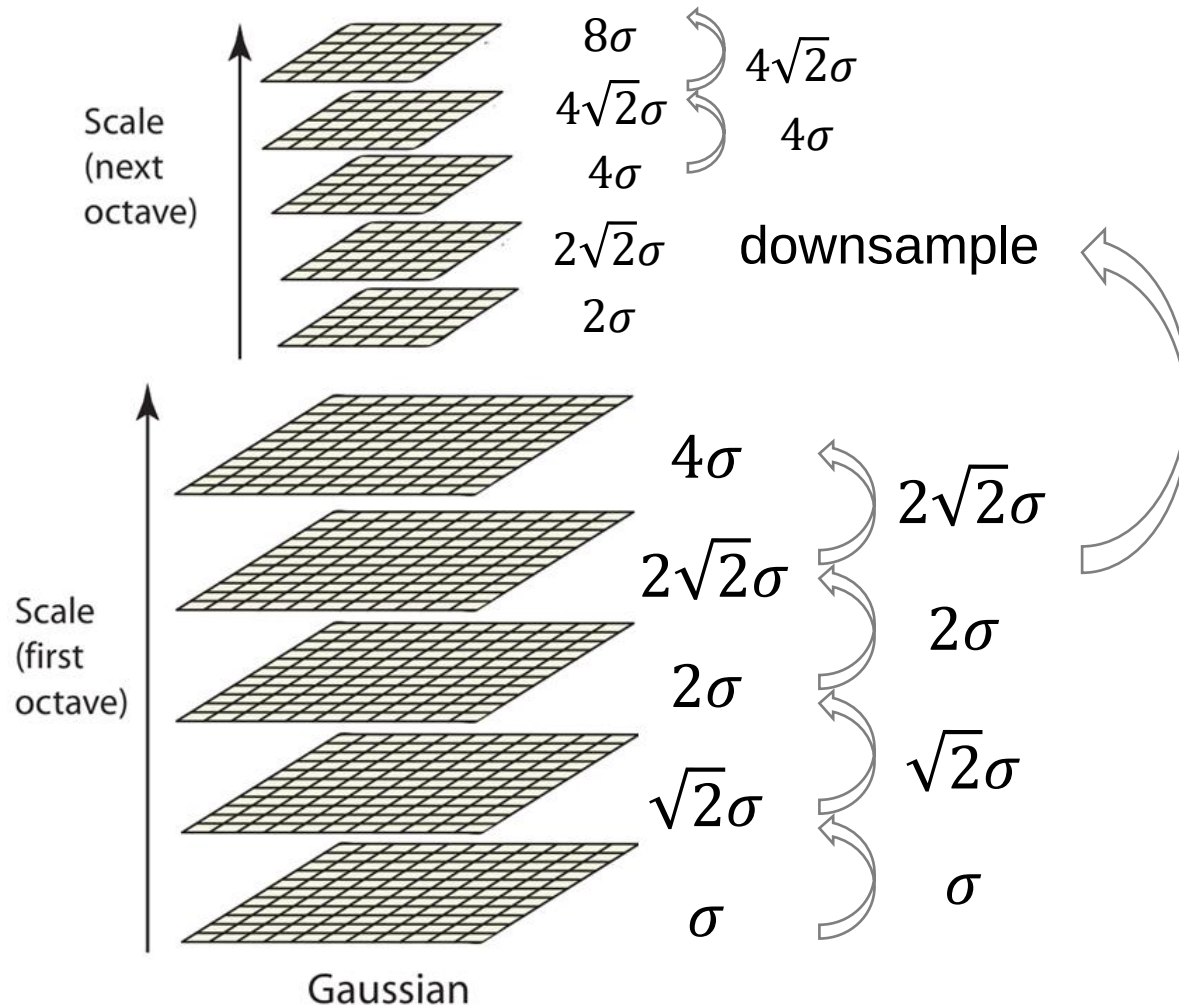
$$f_Y(y) = \mathcal{N}(y; \mu_Y, \sigma_Y^2) = \frac{1}{\sqrt{2\pi}\sigma_Y} e^{-(y-\mu_Y)^2/(2\sigma_Y^2)}$$

$$f_Z(z) = \int_{-\infty}^{\infty} f_Y(z - x) f_X(x) dx$$

$$= \frac{1}{\sqrt{2\pi}\sigma_Z} \exp \left[-\frac{(z - (\mu_X + \mu_Y))^2}{2\sigma_Z^2} \right]$$

$$\sigma_Z = \sqrt{\sigma_X^2 + \sigma_Y^2}$$

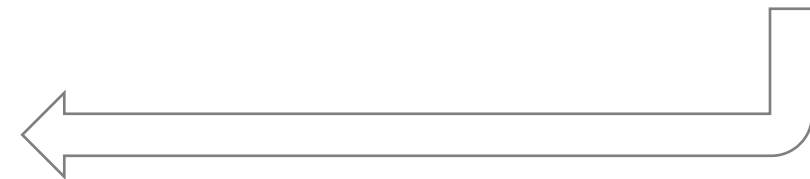
Efficient Construction of Gaussian Pyramid



$$f_Z(z) = \int_{-\infty}^{\infty} f_Y(z-x)f_X(x) dx$$

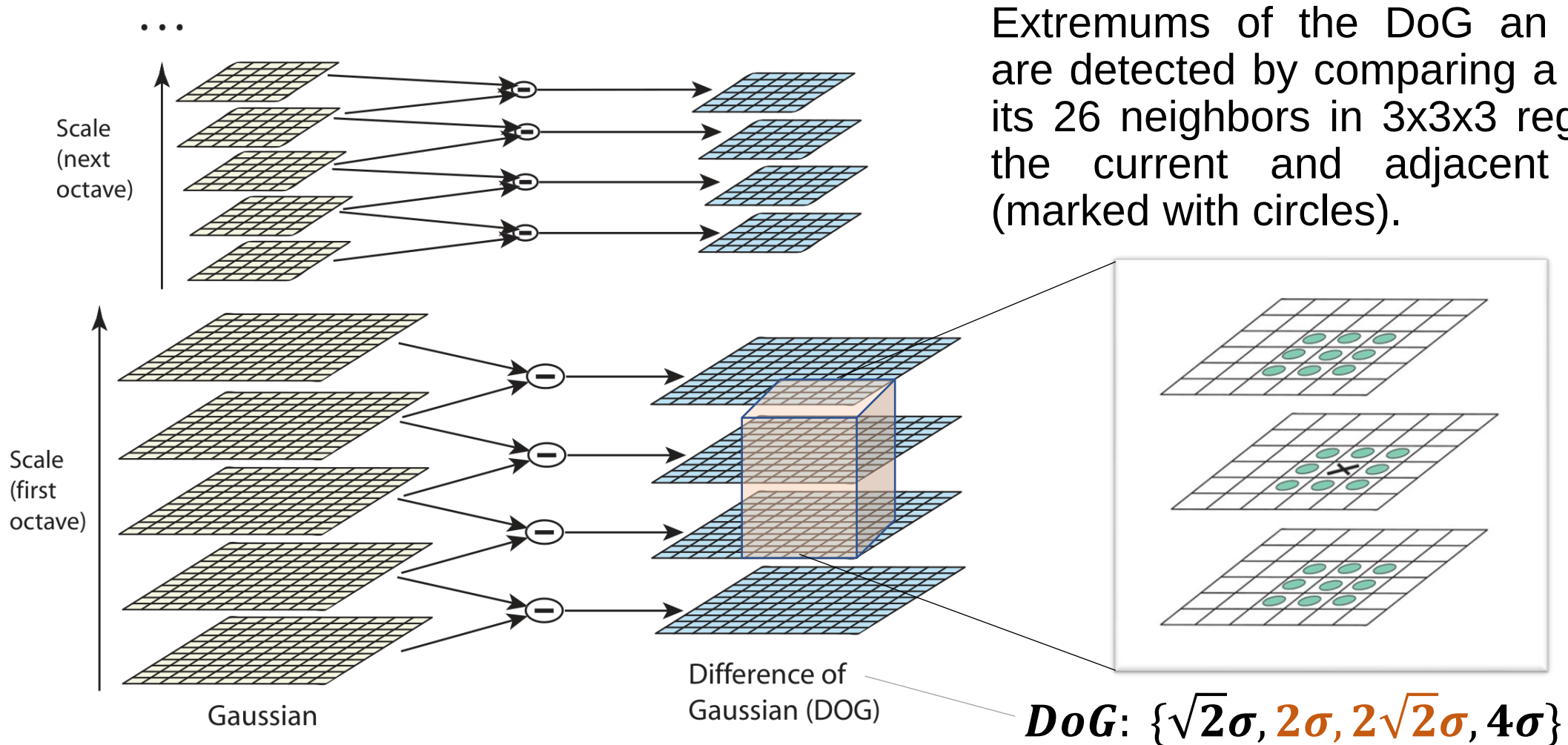
$$= \frac{1}{\sqrt{2\pi}\sigma_Z} \exp\left[-\frac{(z - (\mu_X + \mu_Y))^2}{2\sigma_Z^2}\right]$$

$$\sigma_Z = \sqrt{\sigma_X^2 + \sigma_Y^2}$$



Speed up the Gaussian pyramid with this property

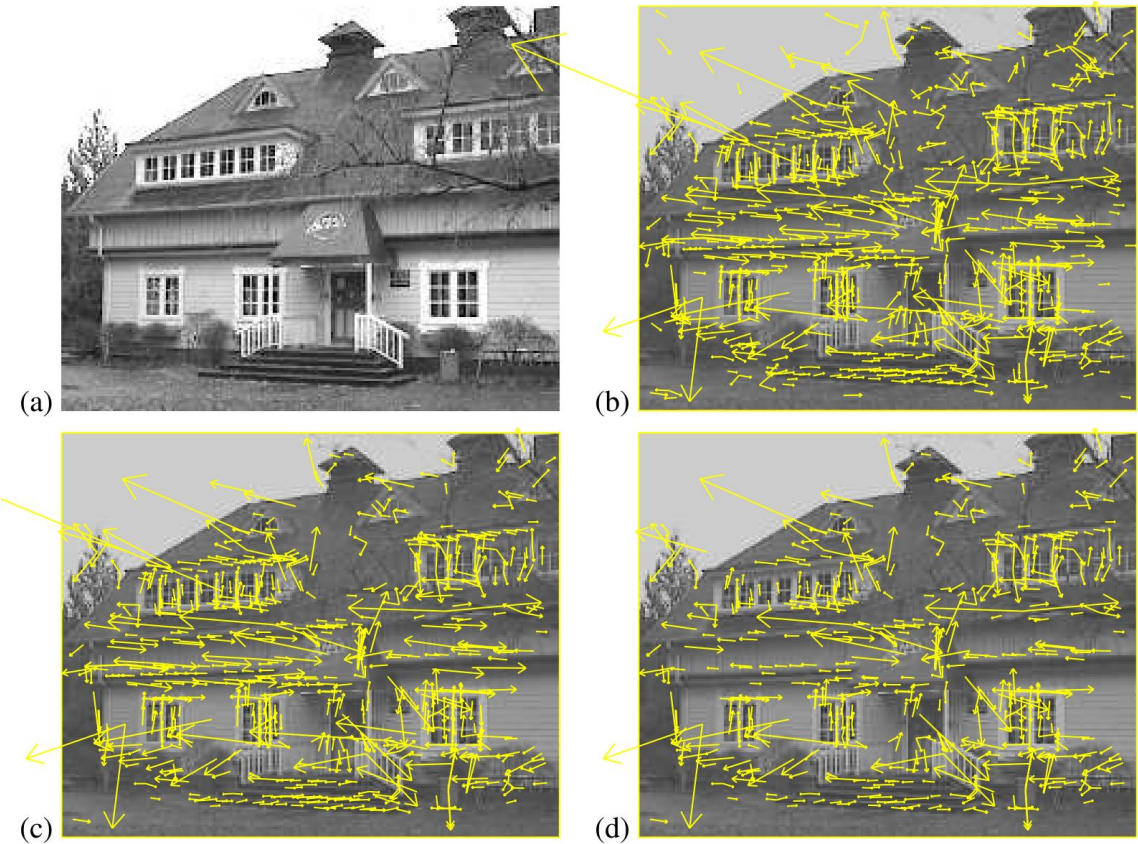
Blob Detection with the Image Pyramid



Extremums of the DoG an images are detected by comparing a pixel to its 26 neighbors in 3x3x3 regions at the current and adjacent scales (marked with circles).

Blob Keypoint Selection

- (a) The original image
- (b) The initial 832 blob locations at maxima/minima of DoG. Blobs are displayed as vectors indicating scale, orientation, and location.
- (c) Applying a **threshold** on minimum contrast, 729 blobs remain.
- (d) The final 536 blobs after another threshold to filter blobs around edges. $\Rightarrow$ **DoG has strong responses near edges.**

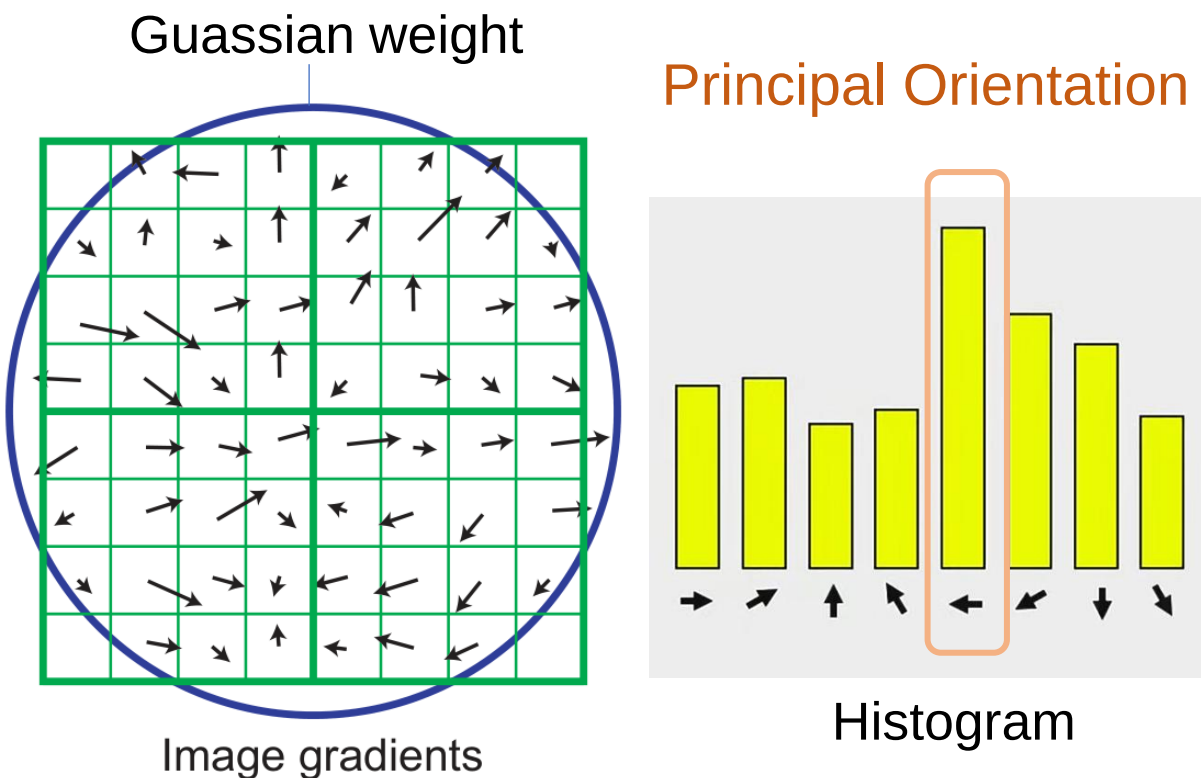


Computing the Principal Orientation of the Feature

- The keypoint scale σ is used to select a smoothed image $L(x, y; \sigma)$.
- Within the region around each keypoint, compute the **gradient magnitude** $m(x, y)$ and **orientation** $\theta(x, y)$. Then construct a histogram of gradient orientations

$$m(x, y) = \sqrt{L_x^2 + L_y^2}, \quad \theta(x, y) = \tan^{-1} \frac{L_y}{L_x}$$

- Peaks in the orientation **histogram** defines the principle orientation.

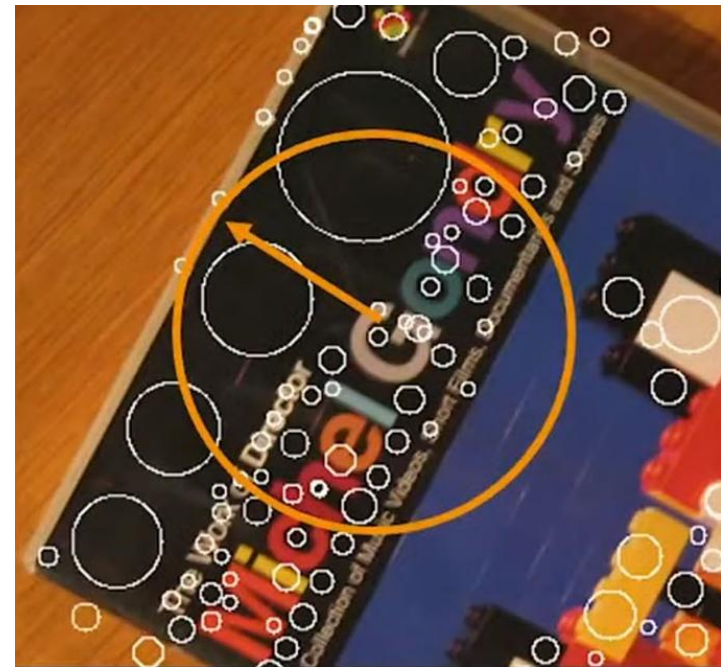
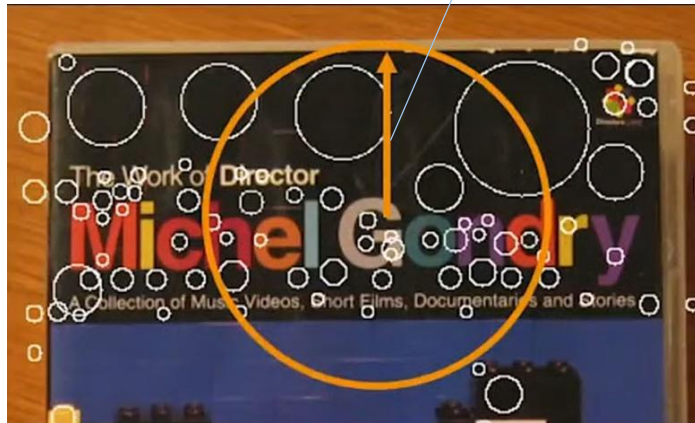


直方图：“研表究明，文字序顺并不定一影阅响读！事证实明当你看这完句话之后才发字现都是乱的。”

Rotation Invariance

- Use the principal orientation to eliminate the rotation

Principal Orientation



Use the Gradient Histograms as the Descriptor

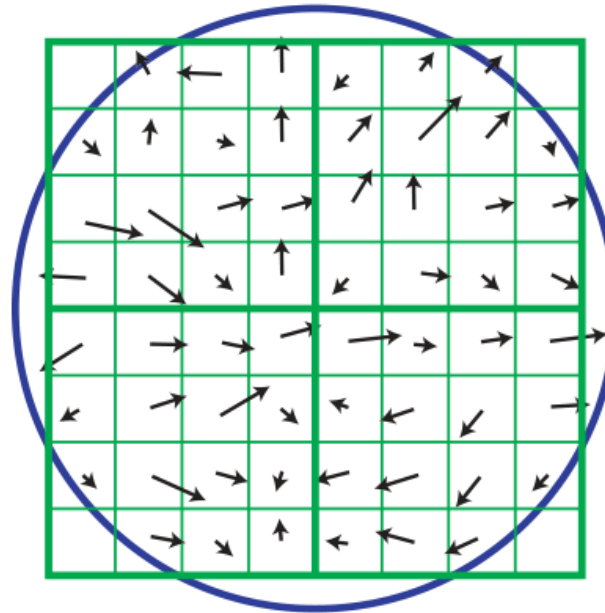
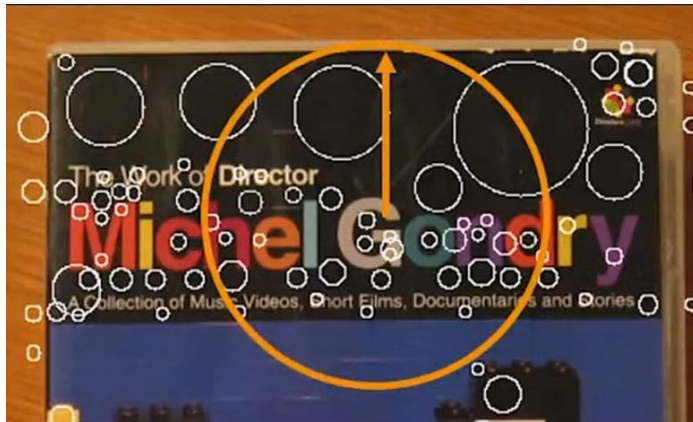
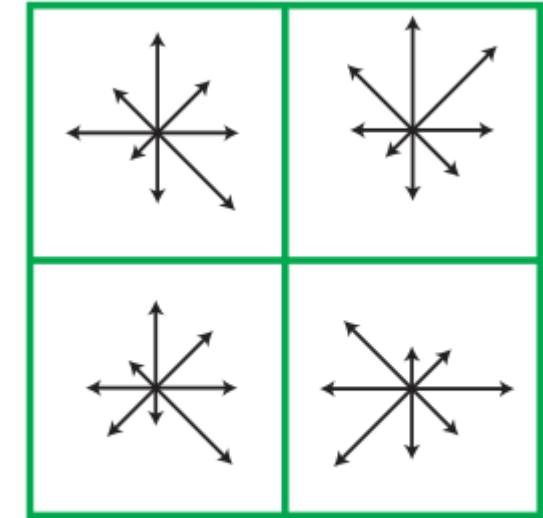
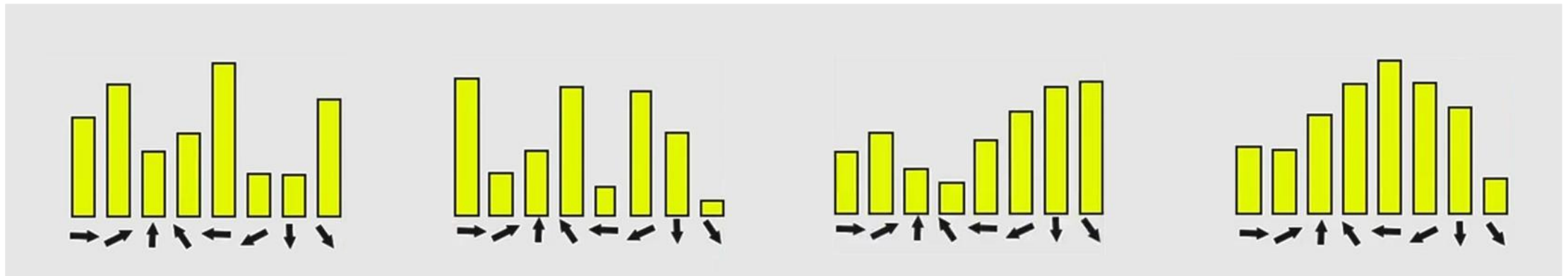


Image gradients



Keypoint descriptor



Use the Gradient Histograms as the Descriptor

Important implementation details

- Use the scale of the keypoint to select the level of the Gaussian pyramid
- Precompute gradients for the pyramid for efficiency
- Use Gaussian window (the blue circle) to avoid sudden changes
- Normalize the feature to unit length to reduce the effects of illuminations
- In practice, use 4x4 local window with 8 orientation bins in each, resulting in a dimension of 128

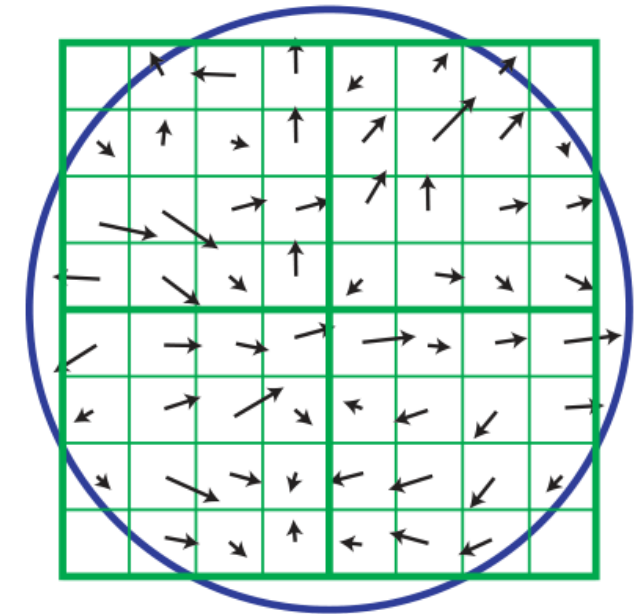
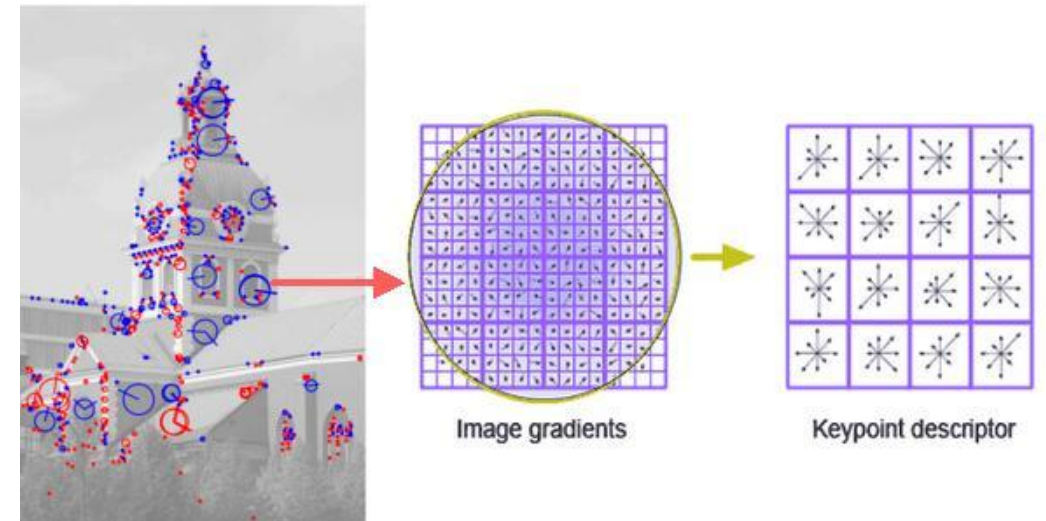


Image gradients

Summary of SIFT

- Blob detection with scale normalization
 - **Scale invariant**
- Align the principal orientation
 - **Rotation invariant**
- Normalized histograms of gradients
 - **Distinctive** (and ignore accurate positions)
 - **Robust to noise and illumination change**
- Leverage image pyramid
 - **Efficient to compute**
- Widely used for Image matching and recognition



Comparing SIFT Descriptors

- Let $H_1(k)$ and $H_2(k)$ be 2 SIFT descriptors (128 dimensions)
- **L2 Distance**: The smaller, the better; Perfect match if $d(H_1, H_2) = 0$

$$d(H_1, H_2) = \sqrt{\sum_k (H_1(k) - H_2(k))^2}$$

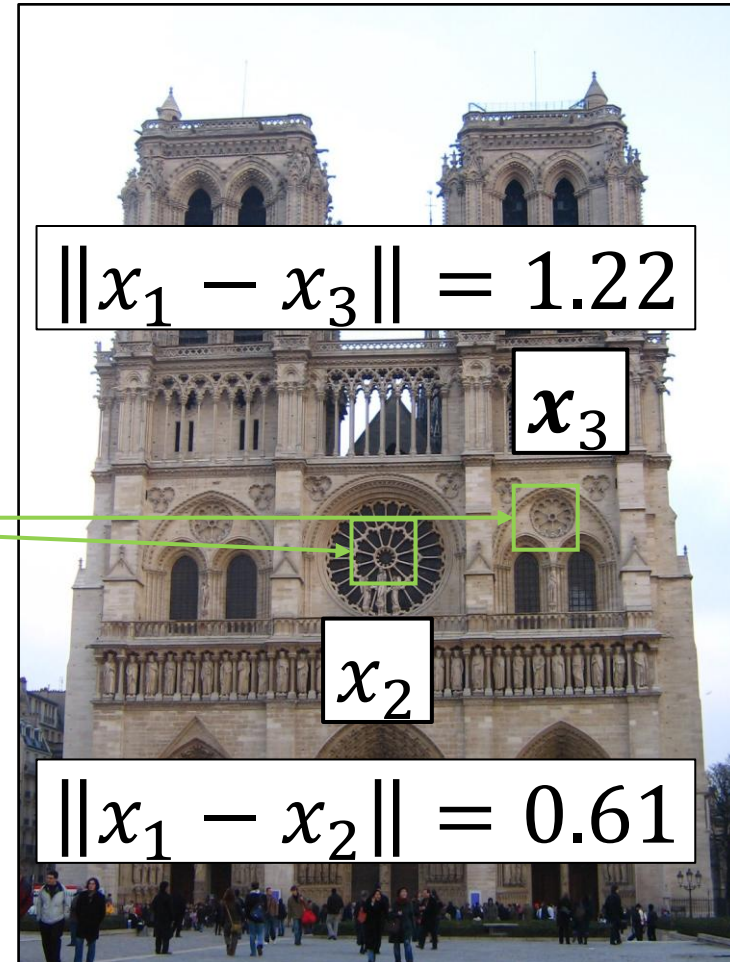
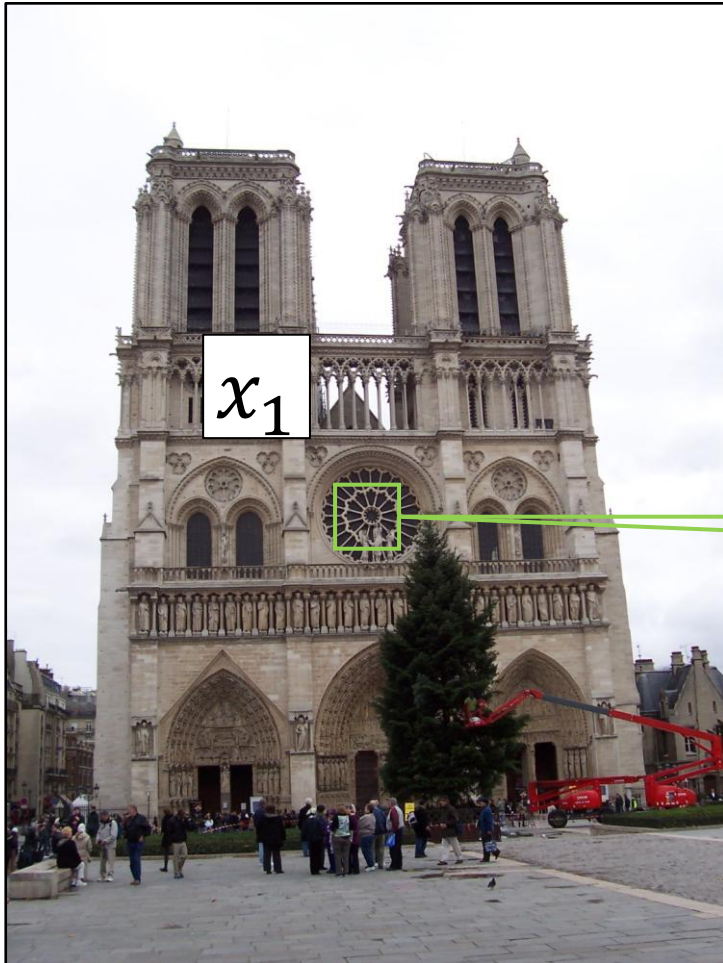
- **Normalized Correlation**: The larger, the better; Perfect match if $d(H_1, H_2) = 1$

$$d(H_1, H_2) = \frac{\sum_k [(H_1(k) - \bar{H}_1)(H_2(k) - \bar{H}_2)]}{\sqrt{\sum_k (H_1(k) - \bar{H}_1)^2} \sqrt{\sum_k (H_2(k) - \bar{H}_2)^2}} \quad \text{where: } \bar{H}_i = \frac{1}{N} \sum_{k=1}^N H_i(k)$$

- **Intersection**: The larger, the better

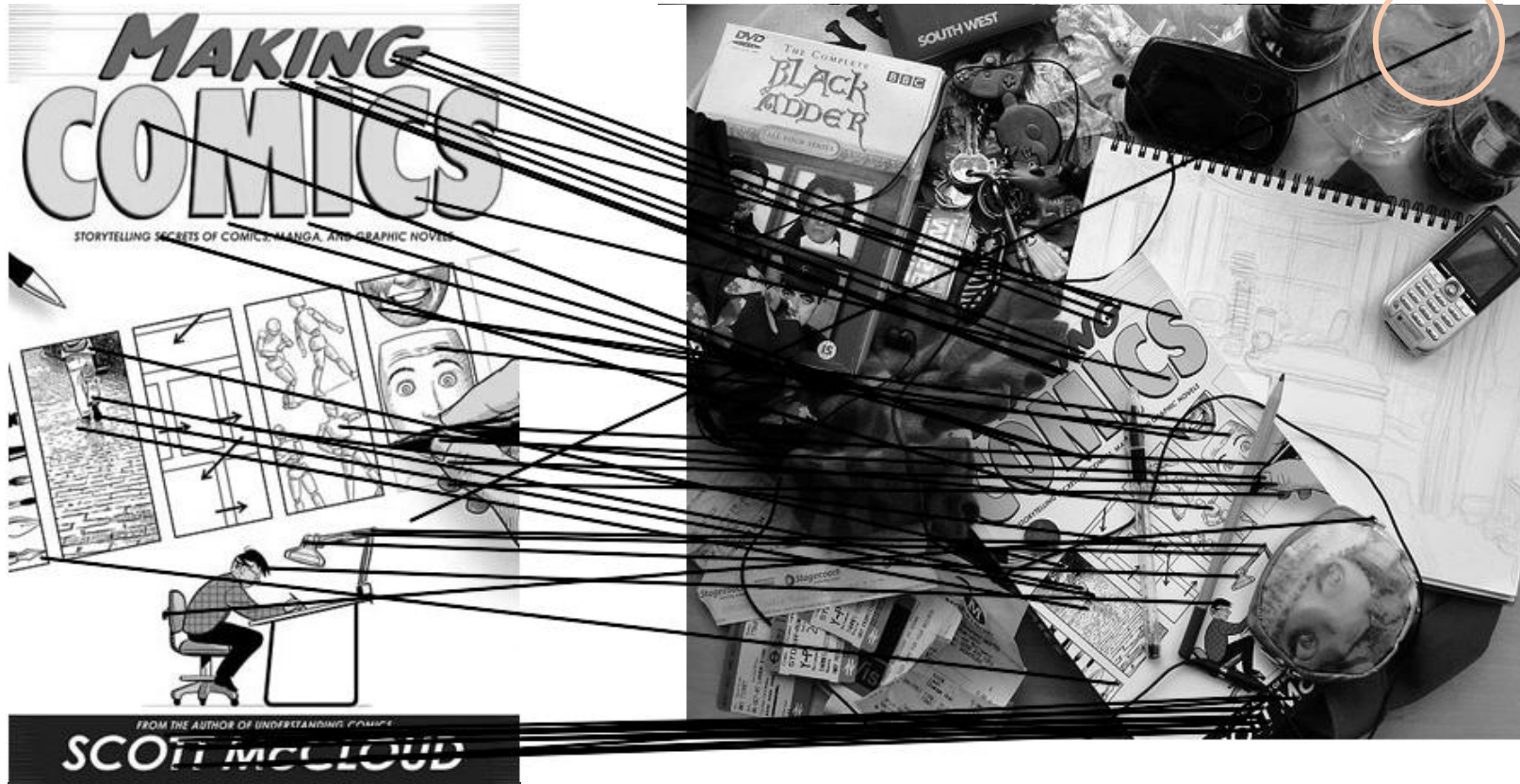
$$d(H_1, H_2) = \sum_k \min(H_1(k), H_2(k))$$

SIFT Feature Matching



SIFT Feature Matching

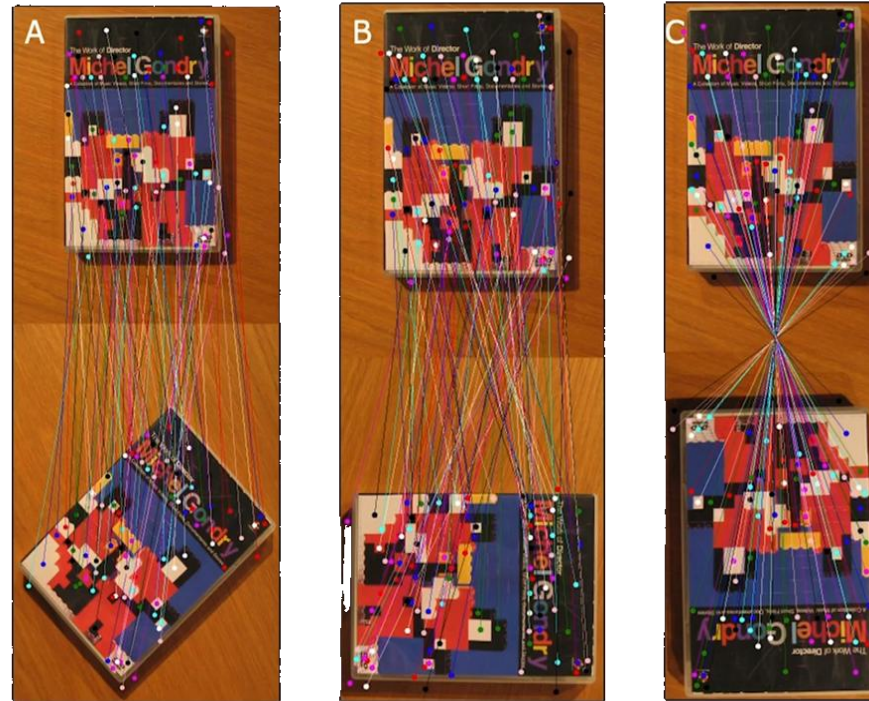
We'll deal with
outliers later



SIFT Feature Matching



Scale Invariant



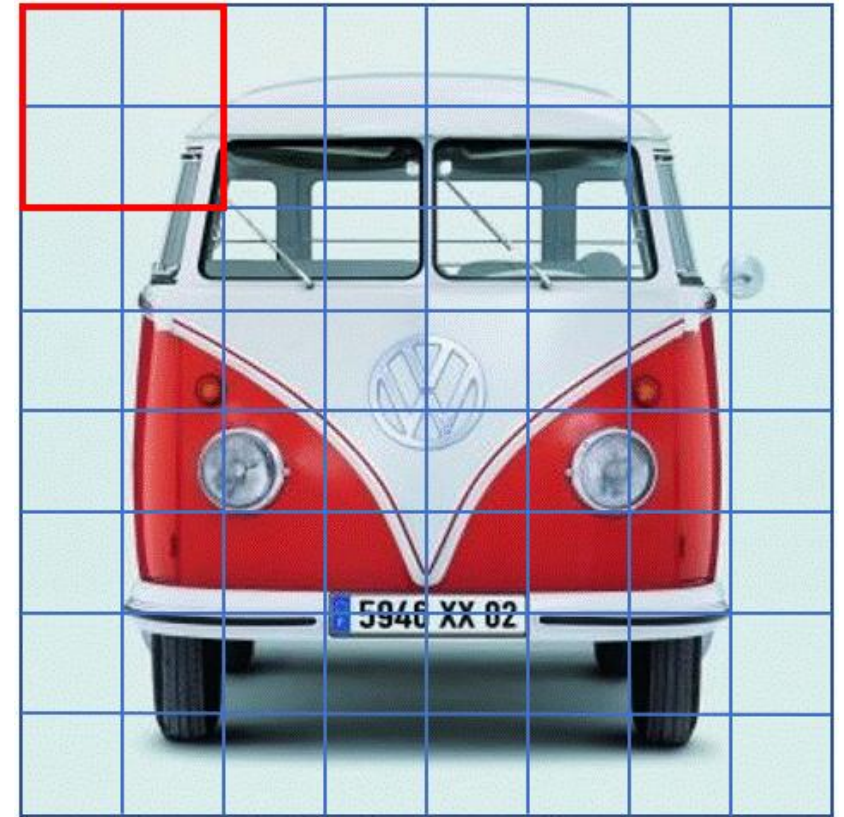
Rotation Invariant



Robust to clusters

Histogram of Oriented Gradients (HOG) Feature

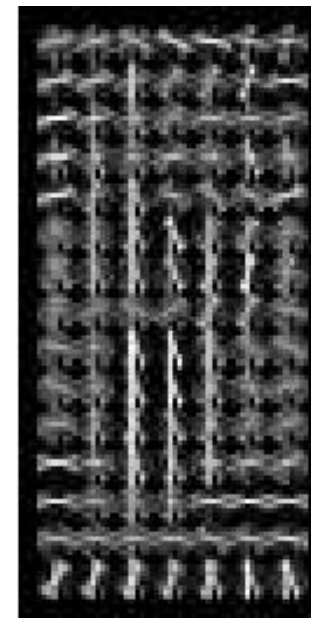
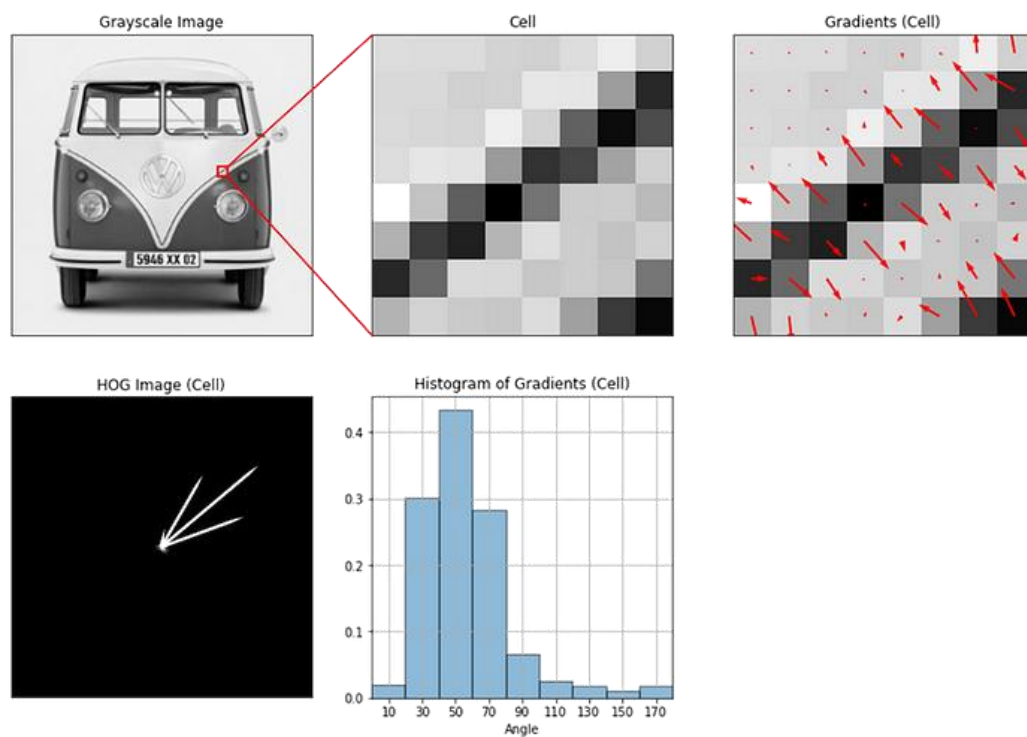
- Local object appearances and shapes can be **characterized with the distribution of gradients** without precise gradient positions.
- 1. Resize the image to 128x64, and calculate the gradients at each pixel.
- 2. The **unsigned** gradients (magnitudes and angles) are divided into 8x8 **cells**, and a 9-point histogram is calculated in each cell.
- 3. **Group** 2x2 **cells** with a stride of 1.
- 4. Normalize the histograms (robust to illumination and shadings)



HOG vs SIFT

- For the computation
 - SIFT: Computed at a sparse set of **scale-invariant key points**, **rotated** to align their dominant orientations
 - HOG: Computed in **dense** grids at a **single** scale **without** dominant orientation alignment (not rotation-invariant)
- For the Usage
 - SIFT: Optimized for matching points across different views or scales, like **image stitching**, **3D reconstruction**, and **object recognition**
 - HOG: Best for **object detection**, especially when the object to be detected has a fairly consistent shape and size, like pedestrians or vehicles

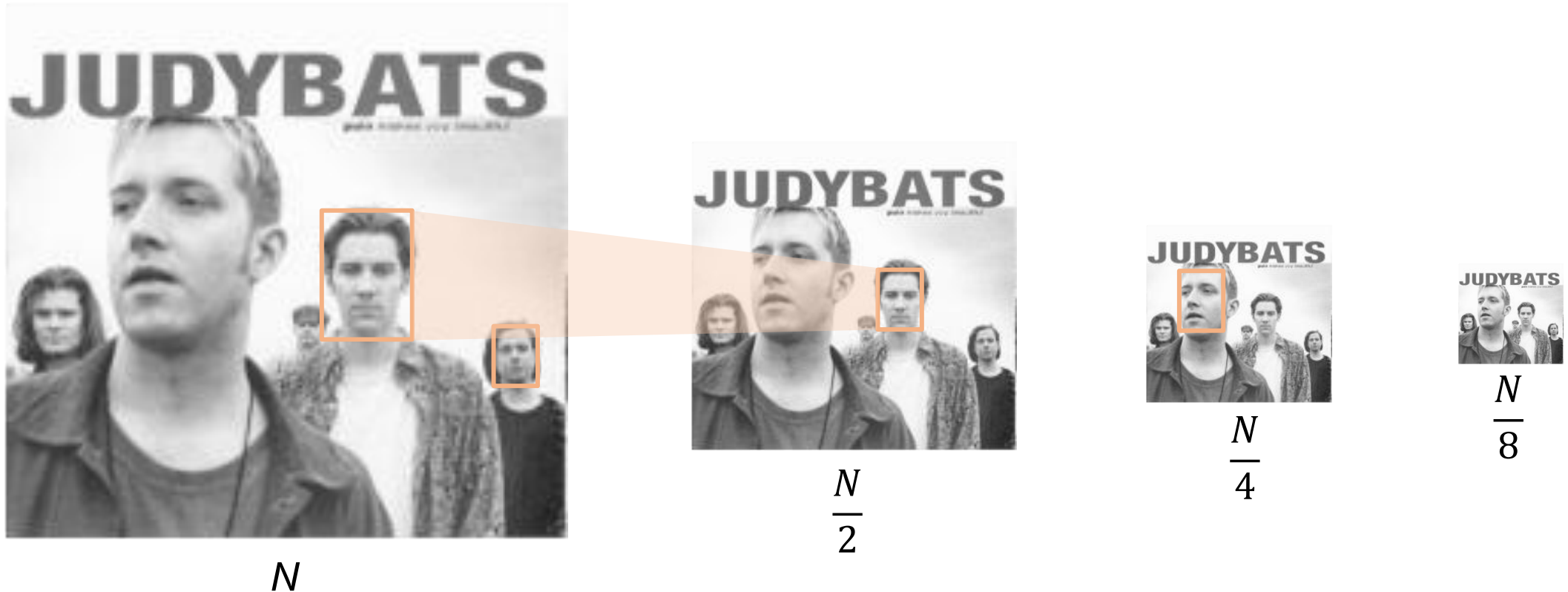
Visualization of HoG Features



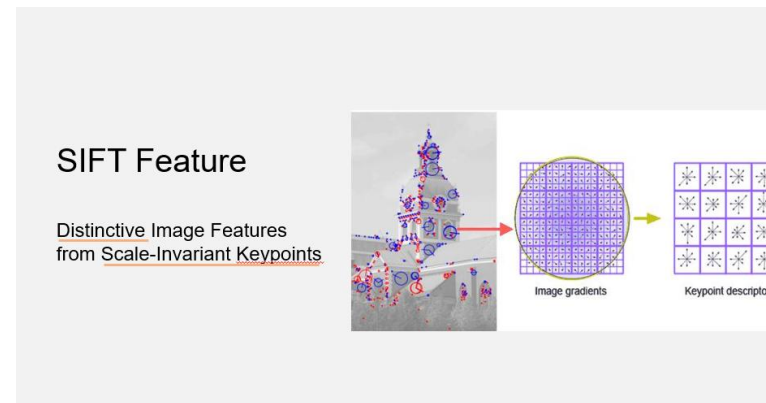
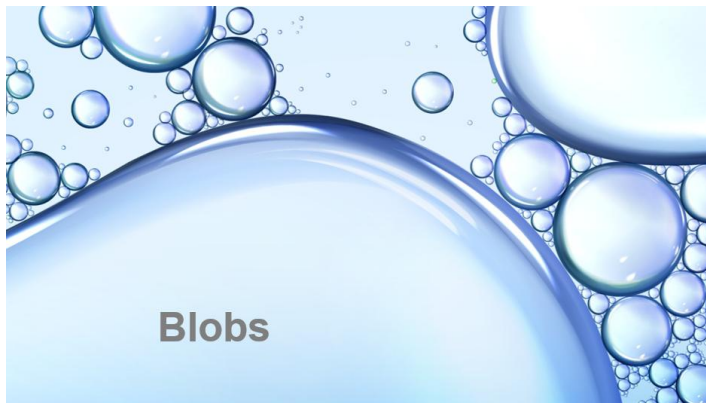
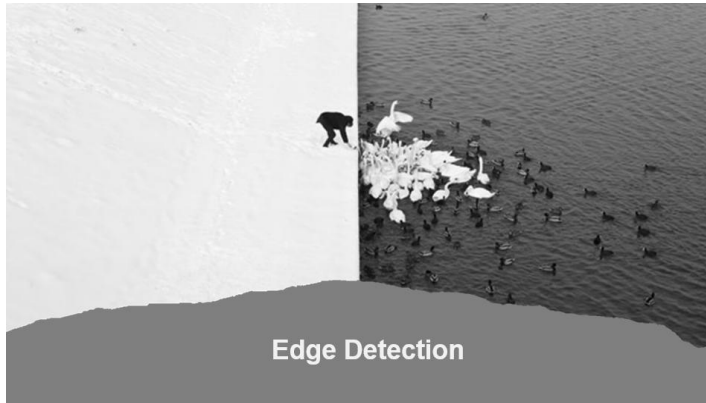
HOG Feature for Detection

M  = training data
 M

- Extract HOG feature and train a linear classifier (SVM)
- Run the classifier with a sliding window with different scales.



Summary



Reference: Chapter 7 of the *Computer Vision* book